

Jankurai

A Versioned Repository Conformance Standard for Trustworthy AI-Assisted Merge Proof Receipts, Merge Witnesses, and Repair Queues for Agent-Native Software Engineering

Jankurai Project • github.com/jeppsontaylor/jankurai

standard_version=0.7.0 • auditor_version=0.7.0 • schema_bundle_version=1.5.0 •
paper_edition=2026.05-ed7



Abstract

AI-assisted coding lowers the cost of producing plausible code but raises the importance of deciding whether a change is safe to merge. Jankurai turns a pull request merge decision into a reproducible, versioned audit artifact. It defines a repository conformance standard for human- and agent-authored changes whose central artifact is the merge witness: a binding among changed paths, owner routing, required proof receipts, observed evidence, score deltas, missing-evidence decisions, tool version, and commit identity.

The paper contributes a stack-neutral, agent-native repository model with owner maps, test maps, generated-zone policy, stable rule identifiers, version manifests, and bounded authority; a conformance mechanism based on proof lanes, audit JSON/Markdown, repair queues, and merge witnesses; and a non-normative reference implementation and reference profile. Conformance is pass/fail against required artifacts. Scores are posture and prioritization signals, not merge approval by themselves.

The claim is deliberately bounded. Jankurai does not prove full program semantics, does not claim final empirical superiority, and does not require its Rust/TypeScript/PostgreSQL reference profile. It is a standards proposal with an early local CLI and a conformance-oriented evaluation path. The thesis is operational: repositories claiming Jankurai conformance need current machine-readable artifacts strong enough to audit the merge decision at merge time.

Index Terms

AI-assisted software engineering, agentic coding, trustworthy merge, repository conformance, proof receipts, policy-as-code, CI/CD, generated code governance, supply-chain security.

TABLE I
AGENT-NATIVE MERGE LOOP.

Step	Repository question
Change proposal	Which paths, contracts, and generated zones changed?
Owner routing	Which owner cells are responsible for those paths?
Proof selection	Which lanes are required by path, risk, and policy?
Proof execution	Which commands ran, with what tool versions and artifacts?
Witness creation	What evidence, missing evidence, score delta, and identity bind this PR?
Reviewer/CI decision	Is the witness sufficient to merge, review, block, or ratchet-fail?
History update	Which receipts and baseline rows remain reusable for future repair?

I. NEW BOTTLENECK: TRUSTWORTHY MERGE

An agent opens a pull request that changes a checkout flow. The diff touches product UI, a generated API client, a schema-adjacent file, tests, and documentation. The hard reviewer question is no longer whether the patch looks plausible. It is: what evidence makes this safe to merge?

That question is the center of Jankurai. The public thesis line, “Humans Were the Bug,” is intentionally blunt, but the standard’s claim is more precise: humans were not the bug because humans wrote software; memory-dependent repositories were the bug. Most repositories were organized around human memory, human patience, human taste, and slow human review. That organization worked only while code arrived slowly enough for teams to compensate with folklore and senior-engineer recall. Agentic coding removes that throttle.

GitHub’s 2025 Octoverse report describes an ecosystem where AI is mainstream in development and TypeScript has become the most-used language on GitHub [1]. DORA’s 2025 research frames AI as an amplifier of existing organizational capability rather than an automatic productivity guarantee [2]. Security evidence is less forgiving: Veracode reports that generated code can be syntactically plausible while still containing known flaws [3], and GitGuardian reports accelerating secret exposure in public GitHub activity, including AI-assisted commits [4], [5].

The correct response is not to slow agents down until the old review process survives. The correct response is to redesign the repository as the verification interface that constrains, proves, and repairs every change before merge. Jankurai turns a pull request merge decision into a reproducible, versioned audit artifact. Its central object is the merge witness:

$$\begin{aligned} \text{merge witness} = & \text{changed paths} + \text{owner route} \\ & + \text{required proof receipts} + \text{score delta} \\ & + \text{missing evidence decision} + \text{commit/tool identity} \end{aligned}$$

Agent-native engineering is not “humans approve whatever the model says.” It is human-directed engineering in which intent becomes bounded authority, changed paths become

proof lanes, proof lanes leave evidence, evidence drives repair or exception expiry, and repeated repairs become reusable primitives. The same machinery must govern human-authored code, because a vague PR from a person and a vague patch from an agent fail in the same place: the repository cannot tell what owns the behavior, which proof applies, or what evidence would make the merge trustworthy.

This paper makes four contributions. First, it defines a repository conformance model for agent-native merge. Second, it specifies a merge-witness artifact that binds changed paths to ownership and proof. Third, it turns missing evidence into stable rule IDs and an ordered repair queue. Fourth, it provides a non-normative reference implementation and reference profile. Stack comparison appears later as one policy-weighted profile analysis; the standard’s primary claim is reproducible merge evidence.

II. DEFINITIONS AND THREAT MODEL

Vibe coding is any development mode where plausible code is accepted without explicit ownership, proof routing, generated-boundary checks, security gates, and repair evidence. A *continuous audit* is the local, pull-request, and CI loop that reruns those checks at merge time; it is not assumed to be a hosted daemon. The current implementation records an appendable, versioned evidence history of proof plans, command receipts, evidence indexes, audit reports, and score-history rows. This paper reserves stronger “append-only ledger” and transparency-log language for future signed receipt work.

The threat model starts with false plausibility. Agents can produce code that looks idiomatic, compiles in isolation, and still violates architecture, security, data ownership, or runtime assumptions. DORA’s amplifier finding matters here: AI improves strong systems and worsens weak ones [2]. The repo must therefore treat generated code as a hypothesis until proof lanes accept it.

The standard does not claim every repository using AI must adopt Jankurai. It claims a narrower and testable thing: a repository claiming Jankurai conformance must expose enough machine-readable structure for the claim to be audited. Conformance is a public interface, not a vibe. A repo that mentions Jankurai in prose but cannot emit current audit and proof artifacts is not conformant.

III. LANGUAGES AS BOTTLENECK COMPRESSION

Programming language history is often told as a progression toward abstraction. That is incomplete. A language is a compression package: it decides which burdens move from human memory into syntax, type systems, runtimes, libraries, tooling, and conventions. Machine code compressed nothing. Assembly compressed opcodes into names. Fortran compressed scientific arithmetic. COBOL compressed business records. C compressed portable systems programming into a thin abstraction over hardware. Java and .NET compressed memory management and enterprise portability into managed runtimes. Python compressed exploration and glue. TypeScript

TABLE II
CORE TERMS.

Term	Operational meaning
Agent-native repository	A repository structured so a human or agent can discover ownership, legal edit scope, proof lanes, and evidence requirements from versioned local artifacts.
Proof receipt	A durable record of a proof command, tool identity, inputs, result, and artifact path.
Merge witness	A versioned artifact binding changed paths, owner route, proof receipts, score delta, missing-evidence decision, commit identity, and tool version.
Changed-path scope	The concrete path inventory and risk class used to select owners and proof lanes.
Owner route	The path-to-owner mapping used to decide who or which policy cell is accountable for review and repair.
Generated zone	An output path repaired by changing the declared source contract or generator and regenerating, not by hand edit.
Score delta	The difference between baseline and current audit posture; useful for prioritization, but not itself conformance.
Repair queue	Ordered findings with rule IDs, owners, proof lanes, rerun commands, evidence requirements, and bounded next actions.
Conformance level	A pass/fail claim that required artifacts and gates for HL1–HL5 are current and reproducible.
Reference profile	A non-normative stack or implementation profile that can satisfy the standard without defining it.

TABLE III
THREAT MODEL FOR AUDITABLE MERGE.

Scope	In this paper	Jankurai control
Assets	Source code, generated artifacts, CI config, proof receipts, audit history, policy files, owner maps, test maps, and generated-zone manifests.	Versioned manifests, generated-zone declarations, stable rule IDs, and report schemas.
Actors	Human maintainer, coding agent, CI runner, malicious contributor, compromised dependency/tool, and stale generated artifact.	Bounded authority, owner routing, source-trust hierarchy, and proof receipts.
Trust boundaries	Local workspace, CI, dependency registry, generated-code boundary, repository policy boundary, and release baseline.	Path scope, generated-zone rules, proof lanes, audit reports, and witness decisions.
In scope	Missing tests, stale generated files, owner bypass, unproven generated-code edits, unsafe migrations, security-relevant config drift, prompt injection, and false-green tests.	HLT rule families, hard caps, repair queue, security/contract/db/web lanes, and missing-evidence decisions.
Out of scope	Proving full semantic correctness, malicious compiler/runtime compromise, and organization-level governance outside the repository.	Explicit limitations, signed-receipt future work, and no claim of total program proof.

compressed large-team JavaScript maintenance by adding a static feedback loop to the browser ecosystem, a direction now strengthened by Microsoft’s native TypeScript 7 effort [6].

The agent-era lesson is that syntax is no longer the scarce resource. A model can imitate syntax. It cannot reliably infer unstated ownership, hidden exception policy, or tribal boundaries. The language and repository must make invalid paths structurally harder.

IV. TECHNICAL PROMISE VERSUS STANDARD GRAVITY

New languages emerge when an existing compression package stops matching the dominant pain. They fracture around runtime ceilings, ecosystem gaps, tooling weakness, migration cost, hiring constraints, interop friction, and fashion that lacks enforcement. A language can be technically excellent and still fail as a default company architecture if it lacks the operational surround: boring deployment, security scanning, observability, package maturity, database migration norms, and enough public examples for agents and humans to learn from.

Julia is the cleanest example of legitimate promise without universal default status. Its multiple dispatch, JIT compilation, and scientific-computing focus attack the two-language problem in numerical work: prototype in a high-level language, then rewrite hot paths in C, C++, or Fortran [9]. That is a real problem, and Julia remains a serious answer for many scientific domains. The adoption lesson is not that Julia is bad. The lesson is that solving an inner-loop expression problem does not automatically solve company-scale product engineering, cloud operations, security review, hiring, packaging, and cross-language contract discipline.

The shelf is not a graveyard of bad ideas. It is evidence that technical elegance and default-standard status are different things. Agent-native engineering rewards stacks where examples, tools, contracts, tests, and deployment conventions are abundant enough that agents can be constrained rather than merely prompted.

TABLE IV
LANGUAGE EVOLUTION AS BOTTLENECK COMPRESSION.

Era or family	Human bottleneck compressed	Agent-era interpretation
Machine code and assembly Fortran and COBOL	Remembering opcodes, addresses, registers, and jumps. Expressing domain work without writing every machine operation.	Exactness without reviewability is not enough; agent patches need semantic structure. Domain vocabulary matters because agents need stable concepts to route changes.
C and Unix systems programming	Portable control over hardware, memory, and operating-system interfaces.	Power without structural memory safety increases audit burden; CISA/NSA guidance pushes teams toward memory-safe defaults [7].
Java and .NET	Managed memory, deployment portability, enterprise libraries, and tooling.	Mature ecosystems help, but framework surface can hide ownership boundaries.
Python, R, Julia	Exploration, data analysis, notebooks, and scientific productivity.	Excellent for model/data loops; dangerous when unbounded into product truth. Python 3.14 improves runtime options, but does not change the boundary problem [8].
JavaScript and TypeScript	Browser programming, product velocity, and typed frontend maintenance.	Essential product surface; should not become durable backend truth.
Rust and modern safe systems languages	Memory safety, ownership, concurrency discipline, and explicit error handling.	Strong core for generated code because invalid states can be rejected before runtime.

TABLE V
TECHNICAL PROMISE VERSUS STANDARD GRAVITY.

Language or ecosystem	Real promise	Adoption limiter	Agent-era penalty
Julia	Scientific productivity with performance-oriented multiple dispatch.	Broader product engineering ecosystem and deployment defaults lag dominant stacks.	Thinner general-product corpus and more local conventions.
D	Better C++-class systems ergonomics.	Could not displace entrenched C/C++ ecosystems and tooling gravity.	Smaller examples and weaker enterprise defaults.
Nim and Crystal	Python/Ruby-like ergonomics with native compilation.	Smaller communities, fewer boring production patterns.	Agents get fewer canonical examples and fewer tool-enforced boundaries.
Elm	Frontend reliability and no-runtime-exception ambition.	Ecosystem and interop constraints limited broad adoption.	Excellent lessons, but narrow hiring and corpus surface.
Reason/OCaml frontend	Typed functional UI with serious compiler roots.	Tooling and community fragmentation.	Harder for general agents to infer local conventions.
F# and Clojure	High-leverage functional design on mature runtimes.	Strong niches, smaller mainstream product footprint.	Expert-friendly abstractions can become opaque to repair agents.
Haskell and Scala	Powerful type-system and abstraction research translated to production niches.	Complexity and hiring/training friction.	Proof power helps only when conventions are explicit and common.
Elixir/Phoenix	Fault tolerance, supervision, realtime systems on the BEAM.	Specialist fit rather than universal product default.	Wins realtime collaboration, not general stack standardization.

V. VIBE-CODING FAULT TAXONOMY

Vibe coding is not a mood. It is a fault class: shipping code whose correctness depends on plausibility, confidence, or reviewer intuition instead of machine-checkable proof. The risk-priority number below is **RPN = impact × likelihood × detection difficulty**. It is a jankurai policy weight, not a universal incident statistic. Future releases should calibrate it against repair success, escaped defects, security regressions, and review time.

Public evidence supports treating vibe coding as a high-risk production pattern rather than a harmless workflow style. Veracode reports that AI-generated samples introduced risky

security flaws in 45% of tests [3]. GitGuardian reports accelerating hardcoded-secret exposure in public GitHub activity and AI-service leak growth [4], [5]. Stack Overflow’s 2025 survey reports that developers’ largest AI frustration is output that is almost right but not quite, with time-consuming AI-code debugging close behind [10]. OWASP’s LLM Top 10 names prompt injection, sensitive information disclosure, supply-chain risk, improper output handling, and excessive agency as first-order AI application risks [11]. SetupBench and ContextBench add the operational failure modes: agents still struggle with environment bootstrap and useful context retrieval [12], [13].

The v0.7.0 coverage release turns the source mate-

rial behind this taxonomy into a tracked registry. Every parsed row from `tips/vibe_coding/tip1.txt` through `tip5.txt` maps to exactly one canonical issue in `agent/vibe-coverage.toml`. Appendix A renders the generated table: green means detector-backed Jankurai control plus proof/report evidence, yellow means a reviewed control family exists but still needs issue-specific fixture, CI, or governance evidence. The v0.7.0 registry has no red rows.

Highest-priority controls. A jankurai-conformant repo should block merges when a changed path has no proof lane, a public boundary has handwritten contract mirrors, durable truth moves outside Rust/PostgreSQL ownership, security or secret scans are absent for risky changes, agent permissions are broader than the lane requires, user-facing UI lacks rendered proof, or `fallback/TODO/stub` ships without a scoped exception.

Known solutions are stronger for some rows than others. Secret scanning, SCA, contract generation, DB constraints, Playwright screenshots, axe checks, and generated-zone manifests are known controls. Model/prompt drift, AI visual triage, context retrieval failure, and human-label learning are still emerging controls; jankurai treats them as soft findings until teams can attach deterministic evidence. The taxonomy turns insult into audit surface. A finding is useful only when it names the rule, points at exact evidence, gives a bounded repair, and can be serialized into an agent queue. “This feels messy” is not actionable. `HLT-002-GENERATED-MUTATION` in `apps/web/src/generated/client.ts` with source `contracts/openapi/api.yaml` is actionable.

VI. JANKURAI STANDARD AND CONFORMANCE

Jankurai is the control plane and the formal contribution of this paper. It is not a general-purpose claim that all AI repositories should use one stack or one tool. It is a versioned standard for repositories that want to claim agent-native conformance. The standard binds prose, machine-readable maps, generated-zone policy, audit output, CI behavior, proof receipts, repair evidence, and release governance.

The standard’s public primitive is the merge witness. The trustworthy merge equation is:

$$\begin{aligned} \text{merge witness} &= \text{changed paths} + \text{owner route} \\ &+ \text{required proof receipts} + \text{score delta} \\ &+ \text{missing evidence decision} + \text{commit/tool identity} \end{aligned}$$

```
{
  "schema": "jankurai.merge_witness.v1",
  "commit": "9f3a1c4",
  "tool_version": "jankurai 0.6.1",
  "changed_paths": [
    "apps/web/src/checkout/Checkout.tsx",
    "apps/web/src/generated/client.ts",
    "contracts/openapi/api.yaml"
  ],
  "owner_route": ["web", "contracts"],
  "required_receipts": ["web", "contract", "security"],
  "present_receipts": ["contract"],
  "missing_evidence": ["rendered_ux_receipt"],
  "score_delta": { "from": 91, "to": 89 },
  "decision": "merge_blocked"
}
```

Conformance and score are deliberately separate. Conformance is pass/fail against required artifacts for the claimed level. Score is posture and prioritization. A high score without current required receipts is not conformant. A low score can still be useful as an advisory finding, but it **MUST NOT** be represented as merge approval.

The adoption loop is itself part of the standard: `jankurai init` installs the control plane, `jankurai doctor` reports drift, and `jankurai audit` produces the evidence history. A repository that only mentions the standard but does not install or execute it is not conformant. Conformance is not prose: a repository that cannot emit current audit JSON, audit Markdown, proof receipts, and version-bound artifacts has no Jankurai conformance claim to evaluate.

Jankurai borrows standards mechanics without claiming institutional equivalence. Normative requirements use BCP 14-style **MUST**, **SHOULD**, and **MAY** language for clarity [14]. Standard, auditor, schema, and paper artifacts use SemVer-like compatibility signaling where breaking report changes require a major version, additive fields are minor-compatible, and copy or clarification changes can remain patch-level [15]. Current and previous releases follow the SPDX pattern of visible specification history [16]. The OpenAPI distinction between specification text and schema artifacts informs Jankurai’s split between normative prose, JSON schemas, and patch/minor report compatibility [17]. W3C-style maturity states make draft, candidate, implementation evidence, superseded, and obsolete status explicit [18]. Kubernetes-style conformance means certification must rest on runnable tests and reproducible evidence, not branding [19]. OpenSSF Scorecard motivates automated open-source posture scoring as an adoption surface rather than a private checklist [20].

Adoption is progressive rather than binary. The install ladder is `agents` → `score` → `ci` → `full` → `ratchet`. `jankurai init -level agents` installs only the root router, the short standard, the master plan, and thin provider adapters. This is the smallest useful change because agent behavior is shaped before the repository claims anything about conformance. `-level score` adds local scoring manifests: owner map, test map, generated zones, proof lanes, audit policy, standard version, and minimal command recipes. `-level ci` adds observe-mode CI plus security policy stubs and evidence paths; it must not enforce score 85 by default. `-level full` preserves the profile-driven scaffold for teams that want docs, contracts, database placeholders, UX QA policy, security controls, and profile-specific structure in one step. Ratchet enforcement remains a separate CI operation after a baseline has been reviewed.

HL0 through HL5 are the formal standard interface. HL0 names repositories that are unscored or unrouted. HL1 makes the audit visible but advisory. HL2 adds guarded critical caps. HL3 is the first ordinary conformance target: high and critical findings, score floor, and metadata checks block merge. HL4 adds ratchet behavior against regression. HL5 turns release into a contract across audit, tests, security, contracts, DB, e2e, and versions. A repository may run Jankurai commands below

TABLE VI
RANKED VIBE-CODING FAULT TAXONOMY. RPN IS A JANKURAI POLICY PRIORITY, NOT AN EMPIRICAL FREQUENCY CLAIM.

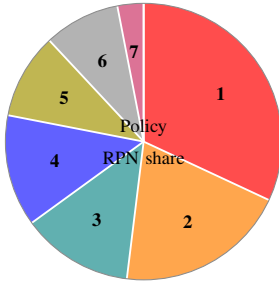
TLR	RPN	Fault	Rule/lane	Required control	Hardness
Business truth	125	False-green business logic	HLT-008 / fast	Domain invariants, property tests, golden workflows, DB constraints, incident replay tests.	Hard
Business truth	100	Authorization or data-isolation drift	HLT-022 / db	Role matrix tests, negative authz tests, owner-map routing, RLS where useful.	Hard
Business truth	80	Idempotency or side-effect duplication	HLT-008 / release	Idempotency keys, replay tests, outbox/workflow proofs, double-charge negative cases.	Hard
Security	80	Plausible insecure generated code	HLT-009 / security	SAST/SCA, secure-code rules, threat-model checklist, negative tests.	Hard
Security	80	Secrets and identity sprawl	HLT-010 / security	Push protection, local/CI secret scans, redacted logs, credential inventory, transcript/artifact scan.	Hard
Security	75	Customer-data or PII leakage	HLT-010 / security	Data classification, prompt/transcript redaction, retention policy, vector-store review, privacy tests.	Hard
Security	75	Excessive agent agency	HLT-012 / security	Least-privilege profiles, destructive-command gates, workspace-only writes, permission receipts.	Hard
Security	75	Prompt injection and hostile context	HLT-011 / security	Source-trust hierarchy, untrusted-content isolation, tool-call validation, no secrets in context.	Hard
Security	60	Improper AI/tool output handling	HLT-011 / security	Schema validation, output encoding, command sandboxing, no direct execution of generated text.	Hard
Security	45	Dependency and supply-chain drift	HLT-016 / security	Dependency rationale, lockfile diff review, OSV/SCA, SBOM, provenance checks.	Hard
Verification	64	False-green tests	HLT-008 / fast	Reviewer-owned acceptance criteria, mutation/fault checks, semantic assertions, red/green evidence.	Hard
Context/setup	64	Context retrieval failure	HLT-015 / fast	Short root router, owner/test maps, local rules, symbol search, command-output filtering.	Soft
Contracts/data	60	Contract and DTO drift	HLT-007 / contract	Generated clients only, contract drift lane, generated-zone lock, compatibility tests.	Hard
Contracts/data	60	Wrong-layer persistence	HLT-006 / db	DB access policy, adapters-only SQL, migration and constraint tests, forbidden imports.	Hard
Contracts/data	60	Destructive migration or data-loss hazard	HLT-021 / db	Rollback/down plan, staged backfill, lock timeout, constraint proof, production rehearsal.	Hard
Context/setup	48	Environment/setup hallucination	HLT-015 / fast	One-command setup, pinned toolchain, devcontainer where useful, setup proof lane.	Hard
Verification	48	Pixel/UI regression	HLT-013 / web	Storyboard states, Playwright screenshots, geometry rules, baseline governance, trace artifacts.	Hard
Verification	48	Accessibility regression	HLT-014 / web	axe/ARIA checks, keyboard journeys, focus/target geometry, expert review for semantic gaps.	Hard
Verification	48	Model, prompt, or eval drift	HLT-008 / ai-eval	Versioned prompts/models, eval baselines, golden datasets, provider-change receipts.	Soft
Repair	45	Observability and repair opacity	HLT-017 / observability	Problem details, stable error codes, OpenTelemetry attributes, correlation IDs, repair hints.	Hard
Entropy	45	Dead-language normalization	HLT-001 / fast	Banned marker scan, scoped allowlist, owner, expiry, exception record.	Hard
Entropy	40	Orphan or dead reachable code	HLT-001 / fast	Reachability checks, owner review, deletion receipts, tests proving replacement path.	Soft
Entropy	36	Performance, memory, or concurrency regression	HLT-018 / release	Benchmarks, query-plan checks, load smoke tests, allocation budgets, tracing.	Soft
Entropy	36	Mega-file or mega-function sprawl	HLT-008 / fast	LOC/function caps, split by owner/use case, focused tests before behavior edits.	Soft
Contracts/data	30	Generated-zone mutation	HLT-002 / contract	Generated-zone manifest, source contract change, regeneration command, drift check.	Hard
Context/setup	30	Documentation or instruction drift	HLT-015 / audit	Canonical manifest, generated mirrors, instruction lint, short root guidance.	Soft

HL3, but it should not market itself as conformant until the required artifacts are current.

MUST, SHOULD, and MAY are used in the RFC sense inside the standard. A repository claiming HL3 or higher MUST provide a root agent router, agent/standard-version.toml, agent/owner-map.json, agent/test-map.json, agent/generated-zones.toml, one-command fast lane, audit JSON, audit Markdown, proof receipts or an evidence index, and an ordered agent_fix_queue. It MUST bind standard_version, auditor_version, schema_version, paper_edition, and target_stack_id in emitted reports. It MUST NOT hand-edit generated zones, hide durable truth in the wrong layer, or merge high-risk changes without a mapped proof

lane. It SHOULD provide SARIF output when the host CI can ingest it, plus JUnit, GitHub summaries, JSONL repair queues, score history, issue exports, and merge witnesses where they improve repair routing.

Stable rule identifiers make the standard repairable. HLT-001 through HLT-027 are stable rule families tied to evidence and repair, not just detector names. They cover dead markers, generated-zone mutation, ownerless paths, unmapped proof, Python product truth, wrong-layer DB access, hand-written contracts, false-green risk, generated-code security, secret sprawl, hostile context, overbroad agency, rendered UX gaps, accessibility gaps, setup/context gaps, supply-chain drift, opaque observability, performance or concurrency drift, streaming runtime drift, CI hardening, destructive migrations, authorization isolation, unsafe input boundaries, agent-tool



#	Share	TLR bucket	Included faults
1	32%	Security	Generated-code flaws, secrets, PII, prompt injection, agency, tool output, supply chain.
2	20%	Business truth	False-green business logic, authorization/data isolation, idempotency.
3	13%	Contracts/data	Contract drift, wrong-layer persistence, destructive migrations, generated mutation.
4	13%	Verification	False-green tests, pixel/UI, accessibility, prompt/model/eval drift.
5	10%	Entropy	Dead language, orphan code, mega files/functions, performance/concurrency.
6	9%	Context/setup	Context retrieval, setup hallucination, instruction drift.
7	3%	Repair	Observability and repair opacity.

Fig. 1. TLR means Top-Level Risk. Shares are computed directly from Table VI: each row contributes its RPN to one TLR bucket, then bucket totals are rounded to whole percentages. The chart is a policy-priority model for jankurai 0.4.0, not an incident-frequency measurement.

TABLE VII
NORMATIVE ARTIFACT CHECKLIST BY CONFORMANCE LEVEL.

Artifact or behavior	HL1	HL2	HL3	HL4	HL5	Requirement
Versioned policy files and stable rule IDs	MUST	MUST	MUST	MUST	MUST	The repository exposes a versioned standard target and machine-readable rule families.
Changed-path inventory, owner routing, generated-zone detection	SHOULD	MUST	MUST	MUST	MUST	Merge decisions bind concrete paths to owners and generated boundaries.
Audit JSON and audit Markdown	MUST	MUST	MUST	MUST	MUST	Machines consume JSON; reviewers consume Markdown.
Required proof receipts and merge witness	MAY	SHOULD	MUST	MUST	MUST	Current high-risk changes cannot merge without witness-bound evidence.
Repair queue, score delta, historical audit comparison	MAY	SHOULD	SHOULD	MUST	MUST	The repo can route missing evidence and detect posture regression.
Reproducible conformance-suite pass	MAY	MAY	SHOULD	SHOULD	MUST	HL5 requires runnable fixtures and independently repeatable expected output.
Signed or independently verifiable receipts	MAY	MAY	MAY	SHOULD	SHOULD	Strongly recommended for release and federation, not required in v0.6.1.

supply chain, release readiness, cost budgets, and human-review receipts. Appendix A expands these IDs into required evidence and repair fields.

The audit loop is intentionally concrete. A failing repository might emit `HLT-010-SECRET-SPRAWL` for an exposed token in a fixture, `HLT-007-HANDWRITTEN-CONTRACT` for a handwritten web DTO, and `HLT-013-RENDERED-UX-GAP` for a checkout change without browser artifacts. The `agent_fix_queue` should order those by TLR priority, route them to security, contract, and web lanes, and require repair receipts: rotated credential and secret-scan output, regenerated client and drift check, screenshot/ARIA/geometry report for the changed viewport.

Proof beats claims. A green score without those receipts is not conformance; it is merely an absence of local evidence.

Centerline drift is the score delta between a repository's claimed standard and its observed behavior. Hard caps are versioned policy, not final empirical truth. The point is to make governance explicit: teams can argue about weights and caps in releases, not in scattered PR comments. Alternate stacks can conform if they provide equivalent owner routing, generated boundaries, proof lanes, security evidence, and repair artifacts; the reference stack is not a loophole around conformance.

`jankurai witness` renders the merge-witness equation as JSON and Markdown for a pull request. It records changed paths, generated-zone touches, required lanes, available receipts, baseline score, new/resolved/carried findings, caps,

TABLE VIII
PROGRESSIVE JANKURAI INSTALL LEVELS.

Level	Installed surface
agents	Root instructions, standard brief, master plan, and provider adapters. No score claim.
score	Agents plus owner, proof, generated-zone, audit, version, and local recipe files for advisory scoring.
ci	Score plus observe-mode workflow, security policy, and stub evidence path. No default score gate.
full	Selected profile unchanged: docs, contracts, DB placeholders, UX, security, and CI where declared.
ratchet	Explicit CI install mode that compares against an accepted baseline and blocks regression.

missing evidence, and a decision: pass, review, block, ratchet fail, or release fail. The slogan is operational rather than decorative: no proof, no merge; no receipt, no trust.

VII. EVALUATION AND CONFORMANCE EVIDENCE

Jankurai is currently best understood as a standards proposal with an early reference implementation, not as a completed empirical benchmark. The evaluation claim in this edition is conformance-oriented: can a repository emit reproducible artifacts that make the merge decision auditable?

A. Current Implementation Evidence

The `jankurai` CLI audits this repository and emits JSON and Markdown reports. It validates rule coverage from `agent/vibe-coverage.toml`, tracks generated zones, owner routes, proof lanes, score history, and repair queues where those controls are implemented, and builds this paper from source. The paper, schemas, agent maps, generated-zone manifest, audit reports, and generated coverage outputs are repository-local artifacts rather than hosted service state.

Current evidence remains limited. Some detectors are deterministic, such as secret-like content, generated-zone mutation, direct DB markers, destructive SQL heuristics, and dead-language markers. Other rows are policy-backed or advisory because they require project-specific fixtures, organization-level proof, or semantic tests that cannot be inferred from repository shape alone.

B. Seed Conformance Suite

This edition includes a seed conformance suite under `conformance/`. It is not a benchmark and does not claim detector completeness. Its purpose is narrower: keep the standard’s rule IDs, expected decisions, and merge-witness outcomes concrete enough for the reference auditor to validate and for future implementations to copy. Kubernetes conformance is a useful precedent because certification rests on repeatable tests and submitted evidence rather than branding alone [19].

C. Research Metrics

The next empirical step is to measure whether the standard improves merge outcomes. Useful metrics include missing-evidence detection, false positives and false negatives, owner-route accuracy, proof-selection accuracy, review time, time-to-repair, CI runtime, regression prevention, security posture change, and token/context reduction. SetupBench and ContextBench show why environment bootstrap and context retrieval deserve direct measurement in agent workflows [12], [13]; Jankurai adds merge-evidence quality as the repository-local outcome to measure.

VIII. NON-NORMATIVE REFERENCE PROFILE SCORING MODEL

This section is non-normative. The Jankurai standard is stack-neutral: a Go, JVM, .NET, Python, Rails, Elixir, or TypeScript-heavy repository can conform if it emits equivalent owner routing, proof receipts, generated-zone evidence, and merge witnesses. The scoring model below is a reference-profile aid for durable product systems maintained by agents and humans together. It deliberately weights correctness, security, contracts, and reparability above first-draft velocity because AI already lowers the cost of plausible drafts. The weights in this edition are policy assumptions, not universal empirical truth.

Points are awarded only for structural properties. “We review carefully” is not a security control. “The senior engineer knows where SQL belongs” is not a boundary. “The README explains it” is not a generated contract. A reference profile should make the wrong path hard to express and the right path cheap to prove.

Hard filters apply before points. A profile with no deterministic local proof lane, no security lane for high-risk changes, no generated contract discipline, or no credible data boundary cannot be recommended even if it is pleasant to write. These caps are policy bands and should be recalibrated against incident data, repair time, false positives, and review burden.

IX. REFERENCE PROFILE COMPARISON

The comparison below is illustrative, policy-weighted, and non-normative. The standard is the merge witness and conformance artifact boundary, not a stack ranking. The scores assume durable product systems with web product surface, API/service behavior, operational security requirements, and long-running maintenance by agents and humans together. Alternative stacks can conform when they produce equivalent proof receipts and merge witnesses.

Sensitivity is concentrated in three weights: memory safety/security, ecosystem corpus strength, and runtime/concurrency. Increase ecosystem weight and Go, .NET, and JVM improve. Increase type-state and memory-safety weight and Rust separates. Increase product velocity and full-stack TypeScript rises, but it hits hard filters unless durable truth, generated contracts, and server ownership are explicit.

TABLE IX
CURRENT JANKURAI CONTROL-PLANE SURFACES.

Surface	Commands	Control-plane role
Adoption and drift	adopt, init, update, doctor	Introduce the control plane without surprise writes, keep adapters and manifests current, and report stale or malformed evidence.
Context and adapters	context-pack, adapters verify/sync, agent verify, hooks install	Turn prompt files and IDE rules into bounded authority tied to owner maps, test maps, generated zones, hooks, and CI entrypoints.
Proof ledger	lane, proof, prove, proof-verify	Route changed paths, execute allowed proof commands, write receipts and evidence indexes, and verify proof artifacts against the current repo state.
Audit, witness, and score ledger	audit, witness, score diff/trend, rules verify, issues export	Emit JSON/Markdown/SARIF/JUnit/summary evidence, merge witnesses, rolling score diffs, ordered repair queues, and machine-readable issue rows.
Security and UX	security run, ux ...	Normalize secret, dependency, SBOM, workflow, and rendered-UX evidence under policy-controlled artifacts.
Repair and expiry	repair-plan, repair, optimize, exceptions expire	Convert findings into bounded repair plans, dry-run or gated apply receipts, optimization proposals, and expiring exceptions.
Cells and public evidence	registry, cell, bench, certify, govern, publish	Promote repeated fixes into reusable cells and publish benchmark, certification, governance, badge, and evidence-bundle artifacts.
Rust proof aids	rust map, rust witness, rust diagnose, explain	Provide symbol maps, witness graphs/diffs, compile diagnostic packets, and local rule explanations for repair agents.

Agent-Native Stack Ranking

ANSS out of 100. Horizontal axis is cropped to $x \geq 85$; higher is better.

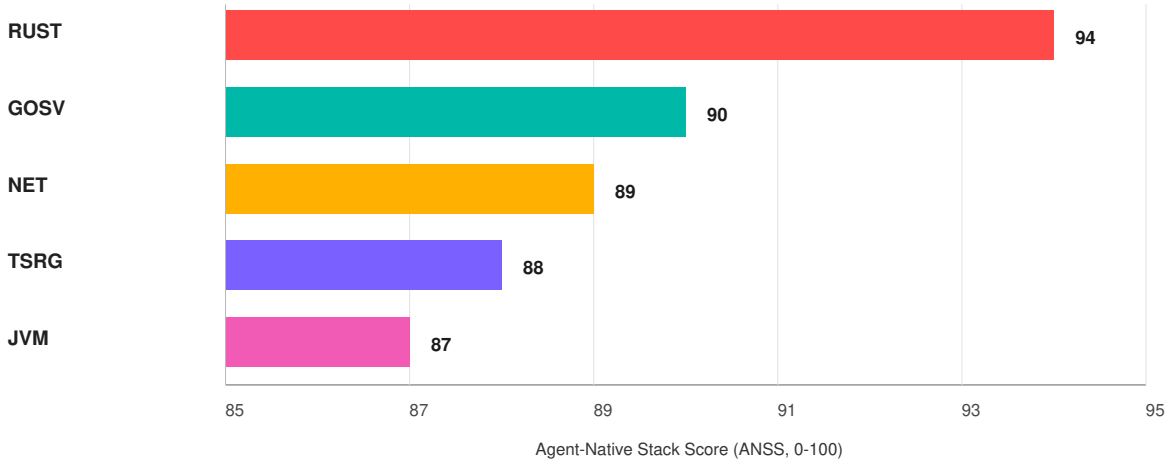


Fig. 2. Reference profile comparison by Agent-Native Stack Score (ANSS), the weighted jankurai scoring model. The figure is a zoomed comparison of close contenders; Table XV gives the policy scores. Labels: RUST = Rust core, GOSV = Go services, NET = C#/.NET, TSRG = TypeScript product plane with Rust/Go compute cells, and JVM = Kotlin/Java. All entries assume TypeScript/React/Vite product surface and PostgreSQL durable truth; streaming buses are workload-specific additions.

These scores are reproducible from stated assumptions, not empirical universals.

A. Streaming Infrastructure: Kafka Today, Rust Tomorrow

Kafka deserves separate treatment because it is both proven and not the baseline. It remains a high bar for durable distributed streaming: it combines replicated logs, high availability, Connect, Streams, client depth, production operating knowledge, and exactly-once processing semantics in the Streams ecosystem [31]–[33]. The jankurai position is explicit: Kafka is a brownfield exception, not stack identity. Use it when the system already needs it, isolate it behind generated event contracts and observable Rust queue adapters, and keep the replacement question measurable.

The roadmap for Rust Kafka-class maturity is therefore a proof program, not a slogan: protocol contract and client golden tests; single-broker produce/fetch; durable log and crash recovery; consumer groups and offsets; replication, controller behavior, and leader epochs; retention and compaction; transactions, security, ACLs, and quotas; then operations, observability, migration tools, and chaos tests. The benefit is not fashion. It is escaping JVM runtime surface where possible, reducing memory and operations cost, preserving Rust-first safety, and giving agents fewer divergent runtime models to reason about.

TABLE X
JANKURAI CONFORMANCE LEVELS.

Level	Meaning
HL0	Unscored: no reliable root instructions or proof lane.
HL1	Advisory: audit runs and emits JSON/Markdown, but findings do not block merge.
HL2	Guarded: critical caps, generated-zone mutation, and missing fast lane block merge.
HL3	Standard: high and critical findings block merge; score floor and conformance metadata are enforced.
HL4	Ratchet: score cannot regress without an exception; repair queue is tracked to closure.
HL5	Release contract: audit, security, contracts, DB, e2e, and artifact versions gate release.

TABLE XI
PUBLIC INTERFACE PROMISES.

Surface	Compatibility promise
Rule IDs	Stable IDs keep old findings and repair receipts interpretable.
Schemas	Breaking output changes require major schema-version movement.
Audit ordering	Findings and queues are deterministic so diffs are reviewable.
CLI exits	Passing, advisory, blocked, and tool-failure outcomes are distinguishable.
Fixtures	Conformance examples include known-good and known-bad repositories.
Generated zones	Output paths declare source, command, and hand-edit policy.
Proof receipts	Receipts name command, inputs, artifacts, result, and tool identity.

X. REFERENCE ARCHITECTURE PROFILE

The Rust/TypeScript/PostgreSQL profile is a reference architecture, not the definition of Jankurai conformance. It gives agents one concrete map, audit one concrete policy, and CI one concrete set of gates for projects that choose this profile. Other profiles can conform when they preserve the same ownership, proof, generated-boundary, security, and witness artifacts.

The filesystem should be a routing table. A patch touching a payment invariant should land in Rust domain/application and PostgreSQL constraints. A patch touching a form should land in TypeScript UI and generated client contracts. A patch touching model behavior should land in the bounded Python service and contract tests. If the agent must infer ownership from convention, the repo has already failed.

Streaming follows the same rule. Event schemas belong in `contracts/`; generated event types belong in declared generated zones; Kafka, Tansu, Iggy, Fluvio, NATS, Redis Streams, or equivalent clients belong in `crates/adapters/queues` or an explicitly declared adapter path. Domain and application code may name business

TABLE XII
SEED CONFORMANCE-SUITE FIXTURES IN THIS REPOSITORY.

Fixture class	Expected signal
HL3 pass	Known-good HL3 shape; expected audit and witness decision: pass.
Ownerless path	HLT-003 blocks an unmapped changed path.
Unmapped proof	HLT-004 blocks a path without a proof route.
Generated drift	HLT-002 blocks generated output changed without source proof.
Secret sprawl	HLT-010 blocks secret-like fixture content.
Destructive migration	HLT-021 blocks destructive SQL without rollback, backfill, lock, and DB proof evidence.
Authz isolation	HLT-022 requires owner/non-owner negative proof.
Input boundary	HLT-023 requires deterministic negative proof for unsafe rendering or input boundaries.
Overbroad agency	HLT-012 blocks tool or agent authority broader than the lane.
Rendered UX gap	HLT-013 requires screenshot, trace, geometry, or accessibility evidence for changed UI.

events, but it should not own broker protocol details.

XI. AGENT REPOSITORY CONTROLS AND TOOL ADAPTERS

Public agent tooling converges on one lesson: instructions matter, but deterministic control surfaces matter more. Codex supports project guidance through `AGENTS.md` [39]. The community `AGENTS.md` specification frames the file as a plain Markdown companion for coding agents [40]. Claude Code loads project memory through `CLAUDE.md` and related hierarchy [41]. Cursor uses project rules with scope metadata [42]. GitHub Copilot supports repository and path-specific custom instructions [43]. These mechanisms should not become divergent manuals. They should be generated or mirrored from one canonical standard.

The adapter problem is an authority problem, not a formatting problem. Jankurai treats prompt files as thin routers over machine-readable policy: `agent/owner-map.json` says who may own a path, `agent/test-map.json` says which lane proves it, `agent/generated-zones.toml` says which outputs are read-only unless regenerated from source, and `agent/standard-version.toml` binds the instruction surface to the standard and paper edition. `jankurai context-pack` turns a task and changed paths into bounded context; `jankurai adapters verify` and `jankurai agent verify` check that provider files still route to the same policy; `jankurai hooks install` and `CI install` commands make the same rules visible before commit and before merge. The result is not a larger prompt. It is a smaller grant of authority backed by files the repo can audit.

Token minimization is not a prompting trick; it is repository design. Large root instructions, duplicate architecture documents, stale docs, broad rereads, and noisy command output waste context and invite contradiction. `ContextBench`

TABLE XIII
NON-NORMATIVE JANKURAI REFERENCE-PROFILE SCORING RUBRIC.

Criterion	Weight	Full-credit signal	Evidence and argument
Agent-verifiable correctness loop	18	Fast type/compile checks, deterministic test routing, generated contracts, property tests, reproducible local and CI lanes.	AI output must be treated as suspect until a proof loop rejects or accepts it; setup and context benchmarks show environment and retrieval reliability are central bottlenecks [12], [13].
Security, memory safety, supply chain	16	Memory-safe core, secret scanning, SCA, SBOM/provenance, lockfiles, unsafe ledger, dependency rationale.	CISA/NSA memory-safety guidance, Veracode GenAI findings, and GitGuardian secrets evidence all push security into structural gates [3], [4], [7].
Runtime performance, memory, cloud cost	13	Efficient hot path, predictable latency, bounded memory, cheap concurrency, low cold-start cost.	Agent-era systems are compute-heavy and parallel; runtime cost becomes architectural, not cosmetic.
Concurrency and resilience	12	Structured async, cancellation, backpressure, idempotency, retries, dead-letter handling, replayable workflows.	Go survey evidence shows broad AI use but persistent quality concerns; concurrency needs guardrails rather than cleverness [21].
Boundary and contract integrity	11	OpenAPI/Protobuf/JSON Schema sources, generated clients, schema drift checks, migration gates.	Cross-language systems work only when contracts are generated and tested; OpenAPI Generator and Buf provide mature paths [22], [23].
Simplicity and review surface	10	Small files, narrow modules, explicit ownership, boring dependency direction, low duplication.	Agents over-edit when boundaries are implicit; SWE-agent and SWE-bench show interfaces and environment shape affect agent results [24], [25].
Ecosystem, hiring, AI corpus strength	8	Large corpus, maintained packages, common deployment patterns, strong model familiarity.	GitHub Octoverse language patterns and TypeScript’s rise matter because public examples are part of the agent substrate [1].
Observability, auditability, provenance	6	OpenTelemetry traces/metrics/logs, request IDs, release provenance, repair receipts.	OpenTelemetry gives a vendor-neutral vocabulary for post-merge repair [26], [27].
Data integrity and durable workflow fit	4	PostgreSQL constraints, migrations, indexes, RLS where useful, transactional workflow safety.	PostgreSQL constraints and row-security policies move durable truth below fallible application code [28], [29].
Product velocity and interop	2	Fast UI loop, generated clients, easy local dev, standard browser ecosystem.	TypeScript 7 and Vite 8 improve feedback speed, but velocity is deliberately weighted below proof [6], [30].

TABLE XIV
REPRESENTATIVE HARD CAPS BEFORE WEIGHTED SCORING.

Missing or unsafe control	Max score
No deterministic local proof lane	65
No security lane on high-risk repo	60
No generated contract discipline for public boundary	80
Direct DB access from wrong layer	66
No secret or dependency scanning in CI	78
Prompt injection or overbroad agent agency in trusted policy	65–78
No rendered UX proof for user-facing web surface	84
False-green tests or destructive migration without safety evidence	70–76

and SWE-Effi support this direction by treating useful context and resource constraints as first-class bottlenecks [13], [44]. RTK-style command filtering is one implementation of the larger principle: show the agent the evidence it needs, not every byte the tool can emit.

The context-pack contract is now budgeted and trust-labeled. A pack carries `-max-tokens`, an estimated token count, included and excluded file lists, and source-trust labels: trusted policy, repo code, generated artifact, proof evidence,

TABLE XV
REFERENCE PROFILE RESULTS UNDER THE JANKURAI SCORING MODEL.

Rank	Stack	ANSS
1	Rust core + TypeScript/React/Vite + PostgreSQL	94
2	Go services + TypeScript/React/Vite + PostgreSQL	90
3	C#/.NET + TypeScript/React/Vite + PostgreSQL	89
4	TypeScript product plane + Rust/Go compute cells + PostgreSQL	88
5	Kotlin/Java JVM + TypeScript/React/Vite + PostgreSQL	87

docs, and untrusted input. The token-saving primitives are concrete: short root routers, path-scoped policy packs, symbol maps, repo maps, filtered command receipts, generated context packs, evidence indexes, stale-doc pruning, and deterministic proof routing that avoids broad rereads. Raw tool output remains governed by policy: trusted policy and repo code are first-class context; untrusted inputs and generated artifacts are

TABLE XVI
REPRESENTATIVE SCORE DECOMPOSITION. BANDS EXPRESS POLICY UNCERTAINTY, NOT STATISTICAL CONFIDENCE.

Stack family	Proof	Sec.	Runtime	Boundary	Ops	Band	Exception logic
Rust + TS/Vite + Postgres	18	16	13	11	36	± 3	Default when no stronger domain constraint exists.
Go + TS/Vite + Postgres	16	13	13	10	38	± 4	Strong cloud-service default where simplicity beats type-state power.
.NET + TS/Vite + Postgres	16	13	12	10	38	± 4	Strong regulated enterprise answer when Microsoft platform fit speeds proof.
Full-stack Next.js + Postgres	13	10	9	8	36	± 6	Excellent product loop; needs stricter backend/data partitioning to avoid truth drift.
Python/FastAPI + Postgres	10	8	8	7	34	± 7	Useful AI/data boundary; risky as durable product core.
Rails + Postgres	11	9	8	7	36	± 6	Fast CRUD app fit; needs generated contracts and stronger service boundaries.
Elixir/Phoenix + Postgres	14	11	11	8	37	± 5	Specialist override for realtime collaboration and supervision-heavy systems.
JVM/Spring or Kotlin + Postgres	15	11	11	10	40	± 5	Brownfield and enterprise fit can outweigh default-stack migration cost.

repo/ apps/ web/ TS/React/Vite UI api/ Rust HTTP/RPC edge crates/ domain/ pure invariants application/ use cases/authz/tx adapters/ Postgres/queues/APIs workers/ jobs/replay/backoff contracts/ OpenAPI/Protobuf/JSON db/ migrations/constraints python/ai-service/ model/eval boundary ops/ CI/security/telemetry	repo/ AGENTS.md short root router agent/ owner-map.json path -> owner test-map.json path -> proof lane proof-lanes.toml runnable commands generated-zones.toml standard-version.toml agent/repo-score.json docs/ decisions/ exceptions/ repair/ runbooks/
--	---

Fig. 3. Ownership spine and agent-control spine for the reference profile.

summarized or excluded when budget is tight.

XII. CONTINUOUS PROOF, CI, AND TEST EXPLOSION

Agent-native testing should be organized by proof question, not author convenience. Domain invariants need Rust unit and property tests. Application workflows need authorization, idempotency, and transaction tests. Adapters need integration and contract tests. PostgreSQL needs migration, constraint, and schema drift checks. The browser needs component tests, rendered UX proof, and Playwright coverage for critical paths.

Public benchmarks and open-source agents show that setup reliability, context retrieval, and interface design are central bottlenecks for autonomous repair [12], [13], [24], [25], [45], [46].

The explosion of tests is useful only if routed. A repo with thousands of tests and no `test-map.json` still asks agents to guess. The standard therefore requires a fast lane for changed files, a contract lane for generated interfaces, a security lane for dependency and secret risk, a DB lane for migrations, a web lane for component and rendered UX proof,

TABLE XVII
RUST-FACING STREAMING CANDIDATES FOR AN AGENT-NATIVE STACK.

Runtime	Strength	Agent-native caution
Kafka	Proven baseline for distributed event streaming, Connect, Streams, client ecosystem, HA, and production operations.	JVM and operations gravity are real; keep it behind adapters rather than letting it define the stack.
Tansu	Strongest Kafka-compatible Rust candidate to watch: stateless broker, Kafka API, PostgreSQL/S3/SQLite storage, schema-backed topics, and Iceberg/Delta/Parquet paths [34], [35].	Not yet a default replacement; ACLs/throttling and S3 compaction/deletion remain explicit gaps in public coverage [35].
Apache Iggy	Strong Apache/Rust-native streaming platform with persistent streams, low latency goals, SDKs, and S3-compatible backup [36].	Greenfield Rust-native option, not a Kafka-wire-compatible drop-in.
Fluvio	Rust/WASM streaming platform positioned as Kafka plus Flink-style processing in one product [37].	Compelling greenfield alternative, not a brownfield Kafka replacement by protocol.
RobustMQ	Ambitious Rust multi-protocol messaging project with Kafka protocol work underway [38].	Kafka support is still incomplete; treat as evaluation material, not replacement proof.

TABLE XVIII
REPRESENTATIVE OWNERSHIP CELLS FOR THE REFERENCE PROFILE.

Cell	Owns	Must not own
apps/web	React UI, route state, forms, local validation, generated API clients, Playwright tests.	Secrets, durable truth, direct SQL, core authorization, handwritten API mirrors.
apps/api	Rust HTTP/RPC edge, request normalization, auth middleware, idempotency keys, response shaping.	Domain invariants hidden in handlers, scattered SQL, UI concerns.
domain crate	IDs, value objects, invariants, state machines, pure decisions, stable error types.	I/O, env reads, network, filesystem, database clients, framework code.
application crate	Commands, authz, transaction orchestration, idempotency, workflow use cases.	Transport details, SQL drivers, UI state, prompt/model logic.
adapters crate	PostgreSQL access, queue/streaming clients, external APIs, filesystem, environment, provider clients.	Domain rules, authorization policy, or event schema ownership.
workers crate	Async jobs, retries, backoff, workflow replay, scheduled tasks.	Duplicated invariants or unowned product truth.
contracts	OpenAPI, Protobuf, JSON Schema, generated stubs and clients.	Hand-edited generated outputs or business logic.
db	Migrations, constraints, indexes, RLS, seed rules, schema snapshots.	Application orchestration or hidden business process.
python/ai-service	Inference, embeddings, evals, prompt experiments, offline transforms, typed AI API.	Product truth, authz, billing, direct production DB ownership.
ops	CI, release, telemetry, secret scanning, SBOM/SCA, provenance, policy gates.	Feature behavior hidden behind manual operations.

an e2e lane for critical browser behavior, and a release lane for full confidence.

CI modes should be explicit. *Advisory* mode emits reports but does not fail merge. *Guarded* mode blocks critical caps. *Standard* mode enforces high and critical findings plus a score floor. *Ratchet* mode prevents score regression without a named exception. *Release* mode gates shipped artifacts on audit, contracts, security, DB, e2e, and version checks.

The continuous audit loop is concrete and local. A changed path enters `agent/owner-map.json` for ownership and `agent/test-map.json` for lane routing. `jankurai lane` or `jankurai proof` turns that route into a proof plan. `jankurai prove` executes allowed commands, writes command receipts and `target/jankurai/evidence-index.json`, and records artifact digests. `jankurai proof-verify` checks the plan and evidence against the current repository state. `jankurai audit` then folds ownership, generated zones, proof receipts, security evidence, UX evidence, and score history into JSON/Markdown/SARIF/JUnit reports.

`jankurai witness` compares the candidate against an accepted baseline and emits the PR proof matrix. Findings become a repair queue through `agent_fix_queue`, `issues export`, or `repair-plan`. This is the real-time part of Jankurai: not an omniscient daemon, but a repeatable local/PR/CI loop that makes merge evidence current.

```
jankurai witness . \
--changed-from origin/main \
--baseline agent/repo-score.json \
--out target/jankurai/merge-witness.json \
--md target/jankurai/merge-witness.md
```

The rolling score is the trust ledger behind ratchet mode. `score diff` compares baseline and candidate reports by score, caps, and finding fingerprint. `score trend` summarizes recent `agent/score-history.jsonl` rows so recurring regression is visible before release governance.

The report itself is proof material. JSON and Markdown are the agent repair surface; SARIF, JUnit, GitHub step summaries, JSONL repair queues, and issue exports are integration surfaces. A failed score with no findings is invalid output: every below-floor dimension must produce at least one routed

TABLE XIX
AGENT TOOL ADAPTER MATRIX.

Tool family	Native surface	jankurai adapter	Non-negotiable rule
Codex	AGENTS.md, local scoped instructions.	Root router plus local ownership files and mapped proof commands.	Do not duplicate durable policy outside the standard.
Claude Code	CLAUDE.md, project memory, local rules.	Mirror root router, version, commands, and generated zones.	Memory may summarize policy but not grant broader permissions.
Cursor	.cursor/rules with scope metadata.	Generate scoped rules from owner map and test map.	Glob rules must agree with owner-map paths.
GitHub Copilot	Repository and path-specific custom instructions.	Short repo-wide mirror plus path instructions for high-risk owners.	No private architecture fork in IDE guidance.
Antigravity-style IDEs	Mission-control UI, terminal/browser approvals, local tasks.	Versioned approval checklist and proof-lane mapping.	Approval policy must respect generated zones and secrets.
Terminal agents	CLI prompt files, repo maps, shell wrappers.	Root router, filtered command output, raw-output escape hatch.	Tool convenience must not bypass CI gates.

TABLE XX
MINIMUM PROOF LANES BY OWNERSHIP CELL.

Layer	Minimum proof
Rust domain	Unit tests plus property tests for invariants and state machines.
Rust application	Integration tests for authz, idempotency, transactions, workflows.
Rust adapters	DB/external integration tests and failure injection.
Contracts	Generation check, drift check, compatibility check.
PostgreSQL	Migration apply, constraint tests, RLS policy tests where used.
TypeScript web	Typecheck, component tests, rendered UX QA, Playwright critical paths.
Python AI service	Contract tests, eval suites, no direct product-truth ownership.
Ops/security	Secret scan, dependency scan, provenance, audit gate.

finding with owner, lane, rerun command, evidence kind, fingerprint, and queue item.

XIII. RENDERED UX AND BROWSER-STEP QA

The browser is a real runtime, not a screenshot afterthought. Agents can create UI that compiles, passes component tests, and still fails through overlapping text, broken responsive layout, invisible controls, stale selectors, missing focus states, blank canvases, cramped buttons, clipped labels, sticky footer collisions, or layout shift. Playwright’s official guidance emphasizes user-visible behavior, resilient locators, isolation, and web-first assertions, which fit agent repair better than brittle sleeps and CSS selectors [47], [48].

The human QA gap is rendered experience. A test can prove that a button exists and calls the correct API; it usually cannot prove that the button is too close to the viewport edge or that a German label breaks a card at 375px. A screenshot diff can prove that pixels changed, but not whether the change is intentional, harmful, or merely anti-aliasing noise. The standard therefore treats rendered UX as a layered

proof system: design-system constraints, generated contracts, deterministic Storybook states, Playwright screenshots, DOM geometry rules, accessibility rules, visual regression, AI/CV triage, and human feedback labels.

The first layer is prevention. Product teams should compose common UI from a constrained design system, not let agents invent raw buttons, modals, menus, forms, color scales, z-index ladders, and spacing systems per task. The W3C Design Tokens Community Group announced a stable 2025.10 design-token specification for sharing design decisions across tools and code [49]. Figma Code Connect now maps design components to real code and feeds those mappings into Figma’s MCP context for AI-enabled tools [50]. This matters because an agent with code-connected design context is less likely to generate approximate HTML/CSS when the repository already owns a real component.

Open-source tools already solve real pieces of the rest. Storybook can turn every story into a visual baseline [51]. Playwright can compare screenshots with `toHaveScreenshot()` and warns that rendering environments must be stable for reliable baselines [52]. Playwright can also compare ARIA snapshots, giving agents a structured accessibility-tree proof instead of pixels alone [53]. Argos and BackstopJS provide visual review and screenshot-diff workflows [54], [55]. Playwright and Storybook can run axe-based accessibility checks, while their docs warn that automation is only a first-line signal [56], [57]. MSW lets Storybook stories use deterministic network responses rather than live API luck [58].

The right open-source posture is composition: use those tools for capture, baselines, ally, mocks, and review, while jankurai owns the policy layer they do not share. The missing core library is a repo-local geometry oracle: a deterministic rule set over DOM boxes, computed styles, accessibility names, scroll sizes, focus state, and viewport state. In this repo, `@jankurai/ux-qa` is that seed runtime. Its report contract should include route/story identity, browser, viewport, screenshot or crop paths, ARIA snapshot path, rule IDs, selectors, severity, and the merge decision. A UI repair receipt

without artifacts is a claim, not proof.

Some criteria are already standards-backed. WCAG 2.2 target-size guidance defines a 24 by 24 CSS-pixel minimum or sufficient spacing exceptions for pointer targets [59]. Core Web Vitals guidance keeps CLS of 0.1 or less as the good visual-stability threshold [60]. Jankurai should therefore treat target-size failures, clipped required text, horizontal overflow, sticky obstruction, missing focus visibility, severe CLS, and unapproved token violations as hard evidence, not as taste debates.

The feedback loop must also be structured. A human approval should label whether a delta is a true layout bug, intentional visual change, acceptable variance, design-system violation, accessibility defect, copy/content issue, responsive-only issue, browser-only issue, false positive, or product/design review item. Storing those labels with the route, viewport, screenshot crop, DOM selector, rule ID, owner, and expiration turns taste review into training data for future thresholds and exceptions. AI or VLM review belongs after deterministic gates: it may summarize a suspicious crop, but it should not be the sole merge gate for taste.

Pixel QA is first-class but not exhaustive on every pull request. The standard risk-routes it. Critical flows, high-risk UI surfaces, visual diff baselines, payments, auth, admin actions, onboarding, and canvas/3D surfaces need browser-step proof in the PR lane. Full viewport and device matrices belong in nightly or release lanes unless the changed path directly touches those surfaces.

XIV. SECURITY, SUPPLY CHAIN, AND PERMISSIONS

Agentic coding expands the write surface. The security posture must therefore be structural. CISA/NSA guidance pushes memory-safe languages as a way to reduce memory-related vulnerability classes [7]. Veracode’s GenAI security results show syntax success does not imply secure code [3]. GitGuardian’s secret-sprawl evidence shows AI-churned repositories need stronger secret gates, not more trust [4].

Permission policy should be boring. Agents should not receive broad filesystem, network, or credential access merely because a task is urgent. High-risk actions need named lanes and evidence: dependency install, secret material, generated-zone writes, migrations, release artifacts, and browser automation against privileged surfaces. A good permission system makes the safe path fast and the dangerous path visible.

In Jankurai this is carried by `agent/security-policy.toml` and `jankurai security run`. The policy separates enabled, required, and advisory tools; the current workspace treats `gitleaks` as required and dependency scanners as advisory when they are unavailable. The command writes a normalized evidence envelope at `target/jankurai/security/evidence.json`, including the wrapper command, policy snapshot, strict-mode flag, and per-step status when the security script emits structured step rows. Missing tools are not silently green: doctor diagnostics and security evidence record whether a

tool was absent, advisory, failed, or blocking. `-strict` turns policy-blocking security status into a failing lane, which is the least-privilege proof Jankurai wants: the agent may proceed only after the repository can show which security tools ran, which were intentionally advisory, and why the merge gate accepted the result.

XV. EXCEPTIONS, OBSERVABILITY, AND REPAIR RECEIPTS

Exceptions are where teams should pool hard-earned repair knowledge. A normal exception tells a human that something failed. An agent-friendly exception tells a repair system what invariant was protected, why it failed, where to read the rule, and what repairs are usually legal.

The shape should combine language-native errors with standardized machine-readable fields. HTTP APIs should use RFC 9457 problem details for boundary errors [64]. Observability should emit OpenTelemetry exception attributes such as exception type, message, stack trace, and error type [27], [65]. TypeScript can preserve underlying causes with `Error.cause` [66]. Rust should prefer typed errors for domain/application code, using context wrappers only at boundaries where opaque provider detail is acceptable [67], [68].

Every controlled error that can reach logs, API responses, background jobs, or tests should expose a stable name, stable code, purpose, reason, common fixes, documentation URL, safe user message, correlation ID, and source cause when safe. The point is not verbosity. The point is to make a failing test or production trace directly actionable for the next agent.

Repair is a governed loop, not permission for unattended mutation. `jankurai repair-plan` reads audit output and turns findings into ordered, schema-validated candidate repairs. `jankurai repair -dry-run` produces a receipt without touching the repository. Real application is gated behind explicit environment variables, bounded risk, and reviewed flags; git commits, pushes, and draft PR creation require separate opt-ins. `jankurai optimize` can propose token, performance, dependency, or dead-code improvements, but those proposals still need the same owner and proof routing as any other change. `jankurai exceptions expire -strict` closes the other half of the loop: temporary waivers are allowed to exist only with owner, reason, and expiry evidence. Jankurai can prepare a proof-backed branch, commit, and draft PR, but high-risk autonomous mutation and auto-merge remain outside the standard.

XVI. MIGRATION, VERSIONING, AND GOVERNANCE

Teams do not reach the default stack through rewrite fantasy. They move by shrinking ambiguity. First add the audit in advisory mode. Then add root instructions, owner maps, test maps, generated zones, version manifests, and proof lanes. Then generate contracts, isolate database access, move durable truth into PostgreSQL constraints and Rust application/domain code, box Python into AI/data boundaries, and add Playwright coverage for critical product paths. Only after the repo can

TABLE XXI
RENDERED UX QA DETECTORS FOR THE TYPESCRIPT PRODUCT SURFACE.

Detector	Machine signal	Merge behavior
Design-system constraints	Component imports, token use, no raw one-off spacing/color/z-index, Code Connect or equivalent mapping.	Block unapproved design-language drift.
Storybook state coverage	Normal, loading, empty, error, long text, locale, theme, mobile, role, and interaction states.	Require stories for reusable components and key screens.
Playwright visual proof	Component and flow screenshots under pinned browser, font, viewport, timezone, and locale settings.	Require review for protected baselines.
DOM geometry rules	Bounding boxes, computed styles, scroll sizes, target spacing, overlap, clipping, wrapping, nested scrollbars, and fixed/sticky collisions.	Block objective layout failures before human QA.
Accessibility structure	axe/WCAG checks, ARIA snapshots, keyboard paths, focus visibility, form labels, and target size.	Block critical known violations; route ambiguous cases to expert review.
Layout stability	CLS and late-shift evidence from Lighthouse or Web Vitals.	Block severe movement on product-critical pages.
AI/CV triage	Semantic review of ambiguous visual deltas and screenshot crops.	Warn or route to humans; do not replace deterministic gates.
Human feedback loop	Labels for true defect, intentional change, false positive, acceptable variance, missing rule, or design-system bug.	Improve thresholds, baselines, stories, and future rules.

TABLE XXII
BROWSER-STEP QA ROUTING.

Risk	Required proof
Critical user journey	Playwright path with role/text/test-id locators, screenshot, and trace artifact.
Responsive layout	Desktop and mobile screenshots with overlap, clipping, and safe-area checks.
Canvas, map, video, 3D	Pixel nonblank check plus interaction or animation evidence.
Design-system component	Storybook state matrix plus visual and ARIA snapshots for variant classes.
Low-risk copy change	Component or lint proof; full matrix deferred unless release lane catches drift.

route and prove changes should teams accelerate agent-driven migration.

First-hour adoption must therefore be no-write unless the operator explicitly applies a reviewed plan. `jankurai adopt -mode observe` and `jankurai init -dry-run` produce adoption or init artifacts without claiming conformance by accident. Migration analysis carries liability forward instead of hiding it: `jankurai migrate -analyze` records missing tests, direct database risk, contract drift, and stack-distance evidence so a brownfield repo can improve by slices. Public evidence uses the same discipline. `registry` and `cell` expose reusable primitives; `bench`, `certify`, `govern`, and `publish` produce versioned benchmark, certification, governance, badge, and public-evidence artifacts. Those artifacts only matter because `agent/standard-version.toml` binds them to a standard version, auditor version, schema version, paper edition, and target stack.

Streaming migration follows the same governance. Do not begin by replacing Kafka. Begin by hiding event buses behind contracts and adapters, measuring broker readiness with replay and compatibility tests, and making the migration path visible in `agent/boundaries.toml`. Rust-native brokers should win by evidence: protocol compatibility where brownfield systems require it, durable recovery, consumer-group correctness, security controls, observability, and operator migration tooling.

The standard must version quickly. The paper, audit script, agent artifacts, standard documents, generated artifacts, and CI policy should share a manifest. In this repo, `agent/standard-version.toml` binds `paper/jankurai.tex`, `paper/jankurai.pdf`, `paper/jankurai.md`, `docs/agent-native-standard.md`, and `agent/JANKURAI_STANDARD.md`. TeX remains the canonical paper source. The Markdown paper is an agent-friendly companion, not the source of truth. Audit outputs now live under `agent/repo-score.json` and `agent/repo-score.md` so the machine-readable repair surface stays inside the agent workspace.

Governance should separate empirical claims from policy choices. Scoring weights, hard caps, conformance levels, RPNs, stack scores, and rule IDs are versioned policy. Public source claims require citations. New audit output fields require schema versioning. Breaking compliance changes require migration notes and example repairs. Non-breaking text clarifications, added advisory rules, and report-field additions should be labeled differently so teams can reason about compatibility instead of treating every update as a rewrite. The goal is not to freeze one 2026 opinion forever; it is to make the standard itself updatable without letting every repository invent a private dialect.

TABLE XXIII
SECURITY CONTROLS FOR AGENT-NATIVE REPOSITORIES.

Risk	Control	Repair evidence
Prompt injection and hostile context	Source hierarchy: root instructions, local owner rules, trusted docs, then untrusted issue/web text.	Receipt names ignored instruction, trusted rule, and resulting safe action.
Secrets and credentials	Secret scanning, no secrets in UI or docs, redacted logs, no model prompt leakage.	Scanner artifact, redaction test, finding closure.
Dependency sprawl	Lockfiles, SCA, SBOM/provenance, dependency rationale for new packages.	Package diff, risk reason, scanner result [61]–[63].
Sandbox escape or tool overreach	Least-privilege tools, explicit write scope, generated-zone protections.	Changed-path report and policy decision.
Unsafe code and FFI	Rust unsafe ledger, boundary tests, memory-safety review.	Ledger entry, invariant tests, owner approval.
False-green security	Security lane mapped by path and risk, not by agent choice.	CI job link and report artifact.

TABLE XXIV
MINIMUM REPAIR RECEIPT FIELDS.

Field	Purpose
rule_id	Stable jankurai rule or exception ID.
owner	Owning path, team, or layer.
failure	What invariant or proof lane failed.
changed_paths	Exact edit surface.
proof_lane	Command or CI job that accepted the repair.
artifact versions	Standard, auditor, schema, paper, and generated artifact bindings.
next_repair	Bounded follow-up when the fix is partial.

TABLE XXV
CI ADOPTION MODES.

Mode	Merge behavior
Advisory	Emit JSON/Markdown reports and never block.
Guarded	Block critical caps and malformed output.
Standard	Block high and critical findings plus score-floor failure.
Ratchet	Prevent score regression without a dated exception.
Release	Gate shipped artifacts on audit, tests, security, contracts, DB, e2e, and versions.

XVII. RELATED WORK

AI coding benchmarks and agents measure task completion, environment use, and agent-computer interfaces. SWE-bench evaluates real GitHub issue resolution, SWE-agent studies agent-computer interface design, OpenHands provides an open software-agent platform, Aider demonstrates terminal pair programming, and SetupBench and ContextBench isolate setup and context-retrieval failures [12], [13], [24], [25], [45], [46]. These systems evaluate or implement coding agents. Jankurai standardizes the repository-local merge decision those agents must satisfy.

Supply-chain and provenance standards secure artifacts and dependencies. SLSA defines incrementally adoptable supply-

chain levels and attestations; SPDX standardizes software bill-of-materials data; OSV provides vulnerability data and scanner tooling; OpenSSF Scorecard automates posture checks [16], [20], [62], [63]. Jankurai uses similar evidence mechanics, but its boundary is the pull-request merge event: changed paths, owners, proof receipts, missing evidence, and witness decisions.

API and schema standards define contract surfaces. OpenAPI, OpenAPI Generator, Protocol Buffers/Buf, and JSON Schema separate normative contract definitions from generated or validated artifacts [17], [22], [23], [69], [70]. Jankurai treats those contracts as inputs to merge conformance rather than as the whole standard.

Conformance ecosystems show how standards become adoptable. W3C process documents define maturity and review states, and Kubernetes conformance uses repeatable tests and submitted evidence to preserve interoperability across implementations [18], [19]. Jankurai follows that pattern with levels, schemas, fixtures, and stable rule IDs.

Agent instruction surfaces are emerging as repository policy. AGENTS.md, Codex project guidance, Claude memory files, Cursor rules, and GitHub Copilot repository instructions all shape agent behavior [39]–[43]. Jankurai does not replace those files; it routes them through one conformance model so instructions, proof lanes, and merge evidence do not diverge.

UI, accessibility, observability, and proof-output ecosystems provide the receipts. Playwright, Storybook, axe/WCAG practice, visual regression systems, OpenTelemetry, JUnit, and SARIF make rendered behavior, accessibility, runtime evidence, and machine-readable findings reviewable [26], [47], [51], [52], [57], [59], [71]. Jankurai’s distinction is to bind those outputs into a versioned merge witness instead of leaving them as disconnected CI logs.

XVIII. LIMITATIONS AND RESEARCH AGENDA

Jankurai does not prove full semantic correctness. A merge witness can show that changed paths were routed to owners, required proof lanes ran, receipts exist, and missing evidence was handled; it cannot prove every business invariant or exclude malicious compiler, runtime, or organization-level

compromise. Current evidence is conformance-oriented and repository-local. The reference implementation is young, some detectors are advisory or partial, and signed receipts, transparent evidence logs, cross-repo federation, and organizational trust remain future work.

The scoring weights need empirical calibration across real repositories. Agent-friendly exception patterns need comparative studies across stacks. The file-size limits should be tested against repair accuracy, not taste. Tool-specific instruction behavior changes quickly. Public benchmarks still struggle to measure long-running maintenance, security regressions, UI correctness, and production repair quality. Alternate stacks can conform when they provide equivalent proof artifacts and merge witnesses.

The maturity model should therefore be explicit. Draft rules need examples and fixtures. Candidate rules need implementation evidence across more than one repository. Stable rules need schema-backed output, documented repairs, and regression tests. Superseded or obsolete rules need migration notes so old receipts remain interpretable.

The strongest version of the criticism is useful: a standard can become ceremony, and a reference profile can become dogma. Jankurai should be judged by whether it improves repair locality, reduces unsafe merges, shortens review cycles, and produces better evidence. If a team can prove those outcomes with a different stack and a compatible control plane, the profile should be documented and studied.

XIX. CONCLUSION

Agent-native engineering is an adaptation problem. Teams that optimize only for generation will get faster at creating ambiguity. Teams that optimize for verification will get faster at rejecting bad code, localizing mistakes, repairing real failures, and changing architecture without guessing.

The durable contribution is not a stack ranking. It is the merge witness, the proof receipt, and the reproducible merge decision. Jankurai asks a repository to make conformance concrete: changed paths, owner route, required proof, observed evidence, score delta, missing-evidence decision, commit identity, and tool identity. The reference profile is one way to satisfy that standard. The standard itself is stack-neutral: no proof, no merge; no receipt, no trust.

TABLE XXVI
CONCRETE FUTURE WORK FOR JANKURAL.

Lane	Expected artifact
Public conformance suite	Runnable tests, fixtures, expected JSON/Markdown reports, and versioned pass criteria.
Hosted or static evidence registry	Immutable evidence bundles, badges, score history, and current/previous standard metadata.
Signed proof receipts and attestations	Artifact digests, command receipts, provenance, and verification command.
Benchmark corpus for repair locality	Repos, patches, route plans, expected lanes, repair time, and escaped-defect labels.
Token-budget optimizer	Short root routers, path-scoped policy packs, symbol/repo maps, filtered command receipts, generated context packs, evidence indexes, stale-doc pruning, and deterministic proof routing.
Context-pack minimizer	Trust-labeled packs with measured token cost, recall, and false-exclusion rates.
Cross-agent adapter conformance tests	Codex, Claude, Cursor, Copilot, terminal-agent, hook, and MCP adapter fixtures.
UI geometry oracle hardening	Viewport matrices, screenshot crops, ARIA snapshots, layout rules, and false-positive corpus.
Semantic rule detector expansion	Fixtures and negative tests for authz, input boundaries, cost controls, release readiness, and human-review evidence.
Exception libraries	Rust, TypeScript, Python, PostgreSQL, and HTTP problem-detail templates with docs links and repair hints.
Cost-budget and kill-switch tooling	Spend limits, quotas, stop conditions, alerts, and receipt-backed emergency controls.
Reusable cell registry and certification	Cell manifests, install plans, proof lanes, benchmark receipts, and certification policy.

TABLE XXVII
INITIAL JANKURAI RULE IDENTIFIERS.

Rule ID	Required evidence	Default repair
HLT-001-DEAD-MARKER	Path, line, matched term, allowlist decision.	Rename to the real state, implement it, delete it, or create a dated exception.
HLT-002-GENERATED-MUTATION	Generated path, source path, regeneration command, diff context.	Change the source contract or generator and regenerate.
HLT-003-OWNERLESS-PATH	Changed path with no owner-map match.	Add owner-map entry or move code into owned path.
HLT-004-UNMAPPED-PROOF	Changed path with no test-map lane.	Add or select deterministic proof command.
HLT-005-PYTHON-PRODUCT-TRUTH	Python path owning product truth, authz, or production DB writes.	Move durable behavior to Rust/PostgreSQL or box Python as AI/data service.
HLT-006-DIRECT-DB-WRONG-LAYER	SQL or DB client marker in UI/domain/application/Python truth path.	Move data access to adapters or migrations.
HLT-007-HANDWRITTEN-CONTRACT	Mirrored DTO, local fetch wrapper, or hand-authored client drift.	Generate from OpenAPI/Protobuf/JSON Schema.
HLT-008-FALSE-GREEN-RISK	Changed behavior accepted by unrelated, weak, skipped, or mock-only lane.	Route to owner lane and add semantic assertion.
HLT-009-GENERATED-SECURITY	AI-generated code changes auth, input handling, crypto, deserialization, logging, or filesystem behavior without security proof.	Run security lane, add negative tests, and attach threat-model evidence.
HLT-010-SECRET-SPRAWL	Secret-like value, broad env dump, unsafe fixture, prompt transcript leak, or missing secret scan.	Remove secret, rotate if needed, add scanner evidence and redaction tests.
HLT-011-PROMPT-INJECTION	Untrusted issue, web, doc, or tool output changes trusted policy or tool behavior.	Preserve source hierarchy, isolate untrusted context, and record ignored instruction.
HLT-012-OVERBROAD-AGENCY	Agent/tool lane allows broad shell, browser, network, migration, delete, or credential actions.	Add least-privilege permission profile and approval gate.
HLT-013-RENDERED-UX-GAP	User-facing UI change lacks screenshot, trace, geometry, baseline, or viewport proof.	Add Storybook/Playwright/geometry evidence for changed surface.
HLT-014-A11Y-GAP	UI lacks labels, keyboard path, focus visibility, target size, ARIA, or axe evidence.	Add accessibility checks and route semantic gaps to expert review.
HLT-015-CONTEXT-SETUP-GAP	Repo lacks deterministic setup/context routing or agent follows stale surrounding context.	Add one-command setup, owner/test maps, local rules, and filtered output.
HLT-016-SUPPLY-CHAIN-DRIFT	Dependency added without rationale, lockfile review, vulnerability scan, SBOM, or provenance.	Add dependency rationale, scanner result, and provenance evidence.
HLT-017-OPAQUE-OBSERVABILITY	Boundary failure lacks stable code, trace/correlation ID, safe message, or repair hint.	Use problem details, OpenTelemetry attributes, and agent-friendly errors.
HLT-018-PERF-CONCURRENCY-DRIFT	Patch adds polling, query duplication, unbounded concurrency, missing cancellation, or memory risk.	Add benchmark/query/load/backpressure evidence and trace the hot path.
HLT-019-STREAMING-RUNTIME-DRIFT	Queue or streaming client appears outside adapter ownership, or Kafka becomes stack identity without brownfield exception and migration path.	Move broker clients behind queue adapters and event contracts, or document owner, expiry, and migration proof.
HLT-020-CI-HARDENING-GAP	CI workflow permissions, unpinned actions, or missing Jankurai proof posture.	Harden permissions, pin trusted actions, and upload audit/witness evidence.
HLT-021-DESTRUCTIVE-MIGRATION	Destructive SQL under migration paths without rollback, backfill, lock, or DB proof evidence.	Add migration safety evidence or replace with a non-destructive transition.
HLT-022-AUTHZ-ISOLATION-GAP	Authorization, RLS, tenant, or role surfaces lack owner/non-owner negative proof.	Add role matrix, RLS, and cross-user denial tests.
HLT-023-INPUT-BOUNDARY-GAP	Unsafe input boundary, dynamic execution, string SQL, shell, fetch, or rendering sink lacks negative proof.	Add schemas, parameterization, allowlists, sandboxing, and exploit fixtures.
HLT-024-AGENT-TOOL-SUPPLY-GAP	MCP, hook, IDE, extension, or agent rule trust surface lacks supply-chain evidence.	Pin and review tool sources, restrict execution, and separate data from trusted policy.
HLT-025-RELEASE-READINESS-GAP	Launch or production-readiness claim lacks backup, monitoring, rollback, security, and abuse-control artifacts.	Add an artifact-backed release gate.
HLT-026-COST-BUDGET-GAP	Paid or unbounded operations lack budgets, quotas, spend alerts, stop conditions, or kill switches.	Add budget policy and evidence.
HLT-027-HUMAN-REVIEW-EVIDENCE-GAP	Review or proof claim lacks raw logs, receipts, or replayable commands.	Attach reproducible evidence before merge.

APPENDIX A

RULE IDS AND CONFORMANCE EVIDENCE

Stable rule identifiers make audit output durable enough for repair agents, CI annotations, and release notes.

The audit JSON should include the rule ID whenever a finding maps cleanly to one of these rules. Rules outside the initial set may use category IDs until they stabilize.

A. Vibe-Coding Source Coverage

Jankurai 0.7.0 maps every parsed row from `tips/vibe_coding/tip1.txt` through `tip5.txt` into `agent/vibe-coverage.toml`. Coverage is detector-backed only when a narrowed row has deterministic rule coverage plus proof/report evidence; partial when Jankurai has a reviewed control family but still needs issue-specific fixture, CI, or governance evidence. The full row-level registry is supplemental source material in `agent/vibe-coverage.toml`; the generated machine-readable outputs are emitted under `target/jankurai/` by the documented coverage command.

TABLE XXVIII
COMPACT VIBE-COVERAGE REGISTRY SUMMARY FOR V0.7.0.

Status or family	Count	Interpretation
Source rows mapped	260	Every parsed row from <code>tips/vibe_coding/tip1.txt</code> through <code>tip5.txt</code> has one canonical issue.
Detector-backed	17	Narrow issue families with deterministic detector evidence and audit/report artifacts.
Partial	243	Reviewed control families that still require issue-specific fixture, CI, or governance evidence.
Unmapped / none	0	No source rows are intentionally left without a control-family mapping in this release.
Security TLR rows	117	Largest source bucket; includes secrets, prompt injection, agency, supply chain, and input/security boundaries.
Verification TLR rows	61	Includes false-green tests, UI/ally proof, and destructive/release checks that route through proof lanes.
Business-truth TLR rows	24	Includes authorization, data isolation, and state-machine correctness risks.

TABLE XXIX
REPRESENTATIVE VIBE-COVERAGE MAPPINGS.

Source	Issue	Rule	Status	Evidence requirement
<code>tip1:6</code>	Hardcoded secrets	HLT-010	Detector-backed	Secret-like content scan plus audit JSON/Markdown evidence.
<code>tip2:12</code>	Cross-site scripting / unsafe rendering	HLT-023	Partial	XSS payload fixtures, encoding/sanitizer proof, and security lane artifacts.
<code>tip2:28</code>	No rollback / weak backup discipline	HLT-025	Partial	Restore drill, rollback plan, backup evidence, and release gate receipt.
<code>tip2:29</code>	Unsafe migrations	HLT-021	Detector-backed	Destructive SQL heuristic plus DB migration proof evidence.
<code>tip3:47</code>	Prompt-level rules treated as security controls	HLT-012	Partial	Enforced permissions, branch/deploy gates, and agent-tool policy receipts.
<code>tip5:43</code>	Broken business logic / skipped transitions	HLT-008	Partial	State-machine invariants, wrong-actor tests, and proof-lane receipt.

APPENDIX B VERSIONED ARTIFACT MANIFEST

The canonical manifest is `agent/standard-version.toml`. It binds the paper source, rendered paper, agent Markdown companion, full coding standard, and agent standard brief.

```
standard = "jankurai"
standard_version = "0.7.0"
paper_edition = "2026.05-ed7"
auditor_version = "0.7.0"
schema_version = "1.5.0"
target_stack = "rust-ts-vite-react-postgres-bounded-python"
published = "2026-05-04"
```

```
[[artifact]]
id = "paper-source"
path = "paper/jankurai.tex"
kind = "tex-wrapper"
version_field = "paper_edition"
source = "paper/tex/"
command = "just paper"
```

```
[[artifact]]
id = "paper-render"
path = "paper/jankurai.pdf"
kind = "pdf"
version_field = "paper_edition"
source = "paper/jankurai.tex"
command = "just paper"
generated = true
```

```
[[artifact]]
id = "paper-agent-md"
path = "paper/jankurai.md"
kind = "agent-markdown"
version_field = "paper_edition"
```

```
[[artifact]]
id = "coding-standard"
path = "docs/agent-native-standard.md"
kind = "standard"
version_field = "standard_version"
```

```
[[artifact]]
id = "agent-standard-brief"
```

TABLE XXX
TAXONOMY-TO-AUDIT DETECTOR STATUS.

TLR	Rule family	Default lane	Detector status in jankurai 0.7.0
Security	HLT-009 through HLT-024 set	security	Partial: CI scan presence, secret-like content, prompt-injection markers, overbroad agency markers, input-boundary markers, agent-tool supply surfaces, generated-security gaps, CI hardening gaps.
Business truth	HLT-005/008/022	db, fast, release	Partial: Python truth, false-green markers, and authz/data-isolation evidence gaps; domain invariant quality remains evidence-driven.
Contracts/data	HLT-002/006/007/021	contract, db	Implemented for generated-zone risk, handwritten DTO/fetch markers, wrong-layer DB markers, and destructive SQL heuristics.
Verification and UX	HLT-008/013/014/025/026	fast, web, release	Partial: rendered-UX readiness, release gates, cost budgets, and @jankurai/ux-qa; ARIA/axe/baseline governance are evidence requirements.
Context/setup	HLT-003/004/015	fast, audit	Partial: root instructions, owner/test maps, setup/fast lane presence; changed-path coverage is planned.
Repair	HLT-017/027	obs., audit	Partial: structured exception markers, review evidence, and repair hints; production trace quality is evidence-driven.
Entropy	HLT-001/008/018	fast, release	Implemented for dead markers, fallback soup, duplication, file/function size; performance evidence remains soft.
Streaming	HLT-019	contract, obs.	Implemented for direct broker clients outside queue adapters and Kafka brownfield exception shape.

```
path = "agent/JANKURAI_STANDARD.md"
kind = "agent-brief"
version_field = "standard_version"
```

cargo run -p jankurai - versions verifies that VERSION, crates/jankurai/Cargo.toml, CLI constants, standard documents, companion Markdown, artifact bindings, and generated-zone registration agree.

APPENDIX C EXCEPTION AND REPAIR TEMPLATES

Exceptions are allowed only when they are named, scoped, owned, testable, and temporary enough for an agent to repair later.

```
docs/exceptions/<ID>.md

# <ID>: <short name>

Owner: <team or owner cell>
Scope: <exact files, crates, services, contracts, or zones>
Purpose: <what this exception permits>
Reason: <why the normal standard cannot be followed now>
Expiration: <date, milestone, or measurable exit condition>
Allowed edits: <what agents may change without widening scope>
Forbidden edits: <what agents must not touch>
Required proof: <test-map lane names and commands>
Common fixes:
- <first bounded repair pattern>
- <second bounded repair pattern>
Audit behavior: <allowed term/path or score cap policy>
```

```
docs/repair/<YYYY-MM-DD>-<short-id>.md

Rule: <HLT rule id>
Owner: <owner cell>
Failure: <what failed and where>
Changed paths:
- <path>
Proof lane: <command or CI job>
Artifacts:
- agent/repo-score.json
- trace.zip
- screenshot.png
Result: <accepted, rejected, partial>
Next repair: <bounded follow-up or none>
```

APPENDIX D CANONICAL FILE TREE DIAGRAMS

The body uses short diagrams so the reader can see the intended shape without leaving the ranking argument. The expanded diagrams below are the normative target shape for a default jankurai repository. Names may vary, but ownership, proof lanes, generated zones, durable truth, and repair evidence should remain explicit.

```

repo/
  apps/
    web/
      src/
        app/
          components/
            generated/
              stories/
                test/
            playwright/
              storybook/
        api/
          src/
            routes/
              generated/
                auth/
      crates/
        domain/
        application/
        adapters/
          postgres/
          queues/
            external/
        workers/
      contracts/
        openapi/
        proto/
        jsonschema/
      db/
        migrations/
        seeds/
      python/ai-service/
      packages/
        ux-qa/
      ops/
        ci/
        security/
        telemetry/
      TypeScript/React/Vite UI
      routes, shells, providers
      design-system composition only
      API clients and typed contracts
      Storybook state matrix
      Vitest, Testing Library, fixtures
      critical flows and screenshots
      rendered component states
      Rust HTTP/RPC edge
      request decode and response encode
      generated protocol bindings
      session bridge, not domain policy
      pure invariants and policy
      use cases, authz, transactions
      SQLx/queries, migrations, repositories
      jobs, replay, backoff
      third-party clients
      async jobs and projections
      OpenAPI source
      protobuf source
      emitted JSON Schema when needed
      durable schema history
      deterministic test scenarios
      bounded AI/data boundary only
      rendered UX geometry runtime

```

APPENDIX E

COMMAND EXAMPLES FOR ADOPTION AND PROOF

A. Adoption, Update, Doctor

```

jankurai init . --level score --yes
jankurai audit . --mode advisory \
  --json target/jankurai/repo-score.json \
  --md target/jankurai/repo-score.md

jankurai init . --level ci --yes
jankurai ci install . --github --mode observe --dry-run
jankurai ci install . --github --mode observe

jankurai init . --level full --profile rust-ts-postgres \
  --dry-run --plan-json target/jankurai/init-full.json
jankurai init . --level full --profile rust-ts-postgres --yes

jankurai update . --check \
  --out target/jankurai/update/update-plan.json \
  --md target/jankurai/update/update-plan.md
jankurai update . --self --apply --yes

jankurai doctor . --fail-on high \
  --json target/jankurai/doctor.json \
  --md target/jankurai/doctor.md

```

B. Adapters, Context, Hooks

```

jankurai context-pack . \
  --task "repair changed docs and paper" \
  --changed README.md \
  --out target/jankurai/context-pack.json \
  --md target/jankurai/context-pack.md

jankurai adapters verify .
jankurai adapters sync . --ide all --dry-run
jankurai agent verify .
jankurai hooks install . --dry-run

jankurai ci install . --github --mode observe --dry-run
jankurai ci install . --github --mode observe
jankurai ci install . --github --mode ratchet \
  --baseline target/jankurai/baseline-score.json

```

C. Scoring, Proof, and Issue Export

```

jankurai audit . --mode advisory \
  --json target/jankurai/repo-score.json \
  --md target/jankurai/repo-score.md \
  --sarif target/jankurai/jankurai.sarif \
  --junit target/jankurai/jankurai.junit.xml \
  --github-step-summary target/jankurai/summary.md \
  --repair-queue-jsonl target/jankurai/repair-queue.jsonl

jankurai issues export . --format jsonl \
  --out target/jankurai/issues.jsonl

jankurai lane . --changed README.md \
  --out target/jankurai/proof-plan.json \
  --md target/jankurai/proof-plan.md

jankurai proof . --changed README.md \
  --out target/jankurai/proof-plan.json \
  --md target/jankurai/proof-plan.md

jankurai prove . --changed README.md \
  --plan-out target/jankurai/proof-plan.json \
  --plan-md target/jankurai/proof-plan.md

jankurai proof-verify . \
  --plan target/jankurai/proof-plan.json \
  --evidence-index target/jankurai/evidence-index.json \
  --out target/jankurai/proof-verify.json \
  --md target/jankurai/proof-verify.md

```

D. Security, Repair, Migration, and Optimization

```

jankurai security run . \
  --out target/jankurai/security/evidence.json
jankurai security run . --strict \
  --out target/jankurai/security/evidence.json

jankurai repair-plan . --from target/jankurai/repo-score.json \
  --out target/jankurai/repair-plan.json \

```

```

--md target/jankurai/repair-plan.md
jankurai repair . --plan target/jankurai/repair-plan.json \
--dry-run --out target/jankurai/repair-run.json \
--md target/jankurai/repair-run.md

jankurai optimize . --mode token \
--out target/jankurai/optimize.json \
--md target/jankurai/optimize.md

jankurai exceptions expire . --strict \
--out target/jankurai/exceptions.json \
--md target/jankurai/exceptions.md

jankurai migrate . --analyze \
--out target/jankurai/migration-report.json \
--md target/jankurai/migration-report.md

```

E. Cells, Rust Aids, and Publication

```

jankurai registry . \
--out target/jankurai/cell-registry.json \
--md target/jankurai/cell-registry.md
jankurai cell . --cell-id background-job --mode prove \
--out target/jankurai/background-job.json \
--md target/jankurai/background-job.md

jankurai rust map . --out-dir target/jankurai/rust
jankurai rust witness build . \
--out target/jankurai/rust/witness-graph.json
jankurai rust witness diff . \
--old target/jankurai/rust/old-witness.json \
--new target/jankurai/rust/witness-graph.json \
--out target/jankurai/rust/witness-diff.json \
--md target/jankurai/rust/witness-diff.md
jankurai rust diagnose . \
--out target/jankurai/rust/compile-packets.json
jankurai explain HLT-004-UNMAPPED-PROOF

jankurai bench . \
--out target/jankurai/pl2-benchmark-report.json \
--md target/jankurai/pl2-benchmark-report.md
jankurai certify . \
--out target/jankurai/pl2-certification.json \
--md target/jankurai/pl2-certification.md
jankurai govern . \
--out target/jankurai/pl2-governance-policy.json \
--md target/jankurai/pl2-governance-policy.md
jankurai publish . \
--certification target/jankurai/pl2-certification.json \
--benchmark target/jankurai/pl2-benchmark-report.json \
--governance target/jankurai/pl2-governance-policy.json \
--out target/jankurai/public/pl2-public-evidence.json \
--md target/jankurai/public/pl2-public-evidence.md \
--badge-json target/jankurai/public/jankurai-badge.json \
--badge-svg target/jankurai/public/jankurai-badge.svg

```

REFERENCES

- [1] GitHub, “Octoverse: A new developer joins github every second as ai leads typescript to #1,” <https://github.blog/news-insights/octoverse/octoverse-a-new-developer-joins-github-every-second-as-ai-leads-typescript-to-1/>, 2025, gitHub Blog, published 2025; accessed 2026-05-01.
- [2] DORA, “State of ai-assisted software development 2025,” <https://dora.dev/research/2025/dora-report/>, 2025, dORA research page, accessed 2026-05-01.
- [3] Veracode, “2025 genai code security report,” https://www.veracode.com/wp-content/uploads/2025_GenAI_Code_Security_Report_Final.pdf, 2025, official report PDF, accessed 2026-05-01.
- [4] GitGuardian, “The state of secrets sprawl report 2026,” <https://www.gitguardian.com/state-of-secrets-sprawl-report-2026>, 2026, annual report page, accessed 2026-05-01.
- [5] —, “The state of secrets sprawl 2026: Ai-service leaks surge 81% and 29m secrets hit public github,” <https://blog.gitguardian.com/the-state-of-secrets-sprawl-2026/>, 2026, official companion blog summary, published 2026-03-17; accessed 2026-05-01.
- [6] Microsoft, “Announcing typescript 7.0 beta,” <https://devblogs.microsoft.com/typescript/announcing-typescript-7-0-beta/>, 2026, typeScript Dev Blog, published 2026-04-21; accessed 2026-05-01.
- [7] National Security Agency (NSA) and Cybersecurity and Infrastructure Security Agency (CISA), “Memory safe languages: Reducing vulnerabilities in modern software development,” <https://www.cisa.gov/resources-tools/resources/memory-safe-languages-reducing-vulnerabilities-modern-software-development>, 2025, nSA/CISA cybersecurity information sheet, June 2025; accessed 2026-05-01.
- [8] Python Software Foundation, “Python release python 3.14.0,” <https://www.python.org/downloads/release/python-3140/>, 2025, release date Oct. 7, 2025; accessed 2026-05-01.
- [9] J. Bezanson, A. Edelman, S. Karpinski, and V. B. Shah, “Julia: A fresh approach to numerical computing,” *SIAM Review*, vol. 59, no. 1, pp. 65–98, 2017.
- [10] Stack Overflow, “Ai: 2025 developer survey,” <https://survey.stackoverflow.co/2025/ai>, 2025, official developer survey AI section; accessed 2026-05-01.
- [11] OWASP Foundation, “Owasp top 10 for large language model applications v2025,” <https://owasp.org/www-project-top-10-for-large-language-model-applications/assets/PDF/OWASP-Top-10-for-LLMs-v2025.pdf>, 2025, official OWASP project PDF; accessed 2026-05-01.
- [12] SetupBench Authors, “Setupbench: Assessing software engineering agents’ ability to bootstrap development environments,” *arXiv preprint arXiv:2507.09063*, 2025.
- [13] ContextBench Authors, “Contextbench: A benchmark for context retrieval in coding agents,” *arXiv preprint arXiv:2602.05892*, 2026.
- [14] RFC Editor, “Best current practice 14,” <https://www.rfc-editor.org/info/bcp14>, 2026, rFC 2119 and RFC 8174 key words for requirements; accessed 2026-05-04.
- [15] Semantic Versioning contributors, “Semantic versioning 2.0.0,” <https://semver.org/>, 2026, official specification; accessed 2026-05-04.
- [16] SPDX Project, “Spdx specifications,” <https://spdx.dev/use/specifications/>, 2026, official current and previous specification page; accessed 2026-05-04.
- [17] OpenAPI Initiative, “Openapi specification,” <https://spec.openapis.org/oas/>, 2026, official specification site with schema guidance; accessed 2026-05-04.
- [18] World Wide Web Consortium, “W3c process document,” <https://www.w3.org/policies/process/>, 2026, official process document; accessed 2026-05-04.
- [19] Cloud Native Computing Foundation, “Certified kubernetes conformance program,” <https://www.cncf.io/training/certification/software-conformance/>, 2026, official CNCF software conformance page; accessed 2026-05-04.
- [20] Open Source Security Foundation, “Openssf scorecard,” <https://openssf.org/scorecard/>, 2026, official project page; accessed 2026-05-04.
- [21] The Go Programming Language, “Results from the 2025 go developer survey,” <https://go.dev/blog/survey2025>, 2026, survey conducted Sept. 9-30, 2025; results published 2026; accessed 2026-05-01.
- [22] OpenAPI Generator contributors, “Openapi generator usage,” <https://openapi-generator.tech/docs/usage>, 2026, official documentation; accessed 2026-05-01.
- [23] Buf, “Buf documentation,” <https://buf.build/docs/>, 2026, official documentation; accessed 2026-05-01.
- [24] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, “Swe-agent: Agent-computer interfaces enable automated software engineering,” *arXiv preprint arXiv:2405.15793*, 2024.
- [25] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, “Swe-bench: Can language models resolve real-world github issues?” in *International Conference on Learning Representations*, 2024.
- [26] OpenTelemetry Authors, “Opentelemetry documentation,” <https://opentelemetry.io/docs/>, 2025, openTelemetry docs landing page, last modified 2025-08-29; accessed 2026-05-01.
- [27] —, “Opentelemetry semantic conventions,” <https://opentelemetry.io/docs/specs/otel/semantic-conventions/>, 2026, openTelemetry documentation, accessed 2026-05-01.
- [28] PostgreSQL Global Development Group, “Constraints,” <https://www.postgresql.org/docs/current/ddl-constraints.html>, 2026, postgresQL documentation; accessed 2026-05-01.
- [29] —, “Row security policies,” <https://www.postgresql.org/docs/current/ddl-rowsecurity.html>, 2026, postgresQL documentation; accessed 2026-05-01.
- [30] Vite Team, “Vite 8.0 is out!” <https://vite.dev/blog/announcing-vite8>, 2026, vite blog, published 2026-03-12; accessed 2026-05-01.
- [31] Apache Kafka Project, “Apache kafka,” <https://kafka.apache.org/>, 2026, official project documentation; accessed 2026-05-02.
- [32] —, “Kafka streams,” <https://kafka.apache.org/documentation/streams/>, 2026, official documentation; accessed 2026-05-02.
- [33] —, “Consumer rebalance protocol,” <https://kafka.apache.org/42/operations/consumer-rebalance-protocol/>, 2026, official documentation; accessed 2026-05-02.
- [34] Tansu contributors, “Tansu,” <https://github.com/tansu-io/tansu>, 2026, open-source repository; accessed 2026-05-02.
- [35] S. D. Simone, “Tansu aims to provide a kafka-compatible broker written in rust,” <https://www.infoq.com/news/2026/03/tansu-stateless-kafka-compatible/>, 2026, infoQ coverage; accessed 2026-05-02.
- [36] Apache Software Foundation, “Apache iggy incubation status,” <https://incubator.apache.org/clutch/iggy.html>, 2026, apache Incubator status page; accessed 2026-05-02.
- [37] Fluvio Project, “Fluvio,” <https://www.fluvio.io/>, 2026, product documentation; accessed 2026-05-02.
- [38] RobustMQ Project, “Robustmq kafka protocol,” <https://robustmq.com/en/RobustMQ-Kafka/Protocol>, 2026, project documentation; accessed 2026-05-02.
- [39] OpenAI, “Custom instructions with agents.md,” <https://developers.openai.com/codex/guides/agents-md>, 2026, codex documentation, accessed 2026-05-01.
- [40] AGENTS.md contributors, “AGENTS.md,” <https://agents.md/>, 2026, agent instruction file specification and guidance; accessed 2026-05-01.
- [41] Anthropic, “How claude remembers your project,” <https://code.claude.com/docs/en/memory>, 2026, claude Code documentation, accessed 2026-05-01.
- [42] Cursor, “Cursor rules documentation,” <https://docs.cursor.com/en/context>, 2026, cursor documentation, accessed 2026-05-01.
- [43] GitHub, “Adding repository custom instructions for github copilot,” <https://docs.github.com/en/copilot/how-tos/copilot-on-github/customize-copilot/add-custom-instructions/add-repository-instructions>, 2026, gitHub documentation, accessed 2026-05-01.
- [44] SWE-Effi Authors, “Swe-efi: Re-evaluating software ai agent system effectiveness under resource constraints,” *arXiv preprint arXiv:2509.09853*, 2025.
- [45] OpenHands Authors, “Openhands: An open platform for ai software developers as generalist agents,” *arXiv preprint arXiv:2407.16741*, 2024.
- [46] Aider Contributors, “Aider: Ai pair programming in your terminal,” <https://github.com/Aider-AI/aider>, 2026, gitHub repository, accessed 2026-05-01.
- [47] Microsoft Playwright, “Playwright best practices,” <https://playwright.dev/docs/best-practices>, 2026, playwright documentation, accessed 2026-05-01.
- [48] —, “Writing tests,” <https://playwright.dev/docs/writing-tests>, 2026, official documentation; accessed 2026-05-01.
- [49] W3C Design Tokens Community Group, “Design tokens specification reaches first stable version,” <https://www.w3.org/community/design-tokens/>, 2025, community Group announcement; accessed 2026-05-01.
- [50] Figma, “Code connect,” <https://help.figma.com/hc/en-us/articles/23920389749655-Code-Connect>, 2026, official documentation; accessed 2026-05-01.

- [51] Storybook, “Visual tests,” <https://storybook.js.org/docs/9/writing-tests/visual-testing>, 2026, official documentation; accessed 2026-05-01.
- [52] Microsoft Playwright, “Visual comparisons,” <https://playwright.dev/docs/test-snapshots>, 2026, official documentation; accessed 2026-05-01.
- [53] —, “Snapshot testing: Aria snapshots,” <https://playwright.dev/docs/aria-snapshots>, 2026, official documentation; accessed 2026-05-01.
- [54] Argos CI, “Argos visual testing,” <https://argos-ci.com/visual-testing>, 2026, product documentation; accessed 2026-05-01.
- [55] BackstopJS contributors, “Backstopjs,” <https://github.com/garris/BackstopJS>, 2026, open-source repository; accessed 2026-05-01.
- [56] Microsoft Playwright, “Accessibility testing,” <https://playwright.dev/docs/accessibility-testing>, 2026, official documentation; accessed 2026-05-01.
- [57] Storybook, “Accessibility tests,” <https://storybook.js.org/docs/writing-tests/accessibility-testing>, 2026, official documentation; accessed 2026-05-01.
- [58] —, “Mocking network requests,” <https://storybook.js.org/docs/writing-stories/mocking-data-and-modules/mocking-network-requests>, 2026, official documentation; accessed 2026-05-01.
- [59] World Wide Web Consortium, “Understanding success criterion 2.5.8: Target size (minimum),” <https://www.w3.org/WAI/WCAG22/Understanding/target-size-minimum.html>, 2026, w3C Web Accessibility Initiative documentation; accessed 2026-05-01.
- [60] web.dev, “Optimize cumulative layout shift,” <https://web.dev/articles/optimize-cls>, 2025, last updated 2025-02-07; accessed 2026-05-01.
- [61] GitHub, “Working with secret scanning and push protection,” <https://docs.github.com/en/code-security/secret-scanning/working-with-secret-scanning-and-push-protection>, 2026, gitHub documentation; accessed 2026-05-01.
- [62] Google OSV, “Osv-scanner,” <https://google.github.io/osv-scanner/>, 2026, official documentation; accessed 2026-05-01.
- [63] SLSA Framework contributors, “Supply-chain levels for software artifacts,” <https://slsa.dev/>, 2026, official specification site; accessed 2026-05-01.
- [64] M. Nottingham, E. Wilde, and S. Dalal, “Problem details for http apis,” <https://www.rfc-editor.org/rfc/rfc9457>, 2023, rFC 9457.
- [65] OpenTelemetry Authors, “Exception attributes,” <https://opentelemetry.io/docs/specs/semconv/registry/attributes/exception/>, 2026, official semantic conventions; accessed 2026-05-01.
- [66] MDN Web Docs contributors, “Error: cause,” https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Error/cause, 2026, mDN Web Docs; accessed 2026-05-01.
- [67] Rust Project, “Error handling,” <https://doc.rust-lang.org/stable/rust-by-example/error.html>, 2026, rust By Example; accessed 2026-05-01.
- [68] anyhow contributors, “anyhow,” <https://docs.rs/anyhow/latest/anyhow/>, 2026, rust crate documentation; accessed 2026-05-01.
- [69] Protocol Buffers contributors, “Protocol Buffers documentation,” <https://protobuf.dev/overview/>, 2026, official documentation; accessed 2026-05-04.
- [70] JSON Schema contributors, “JSON Schema specification,” <https://json-schema.org/specification>, 2026, official specification page for current and previous drafts; accessed 2026-05-04.
- [71] OASIS, “Static analysis results interchange format version 2.1.0,” <https://docs.oasis-open.org/sarif/sarif/v2.1.0/sarif-v2.1.0.html>, 2023, oASIS Standard; accessed 2026-05-04.